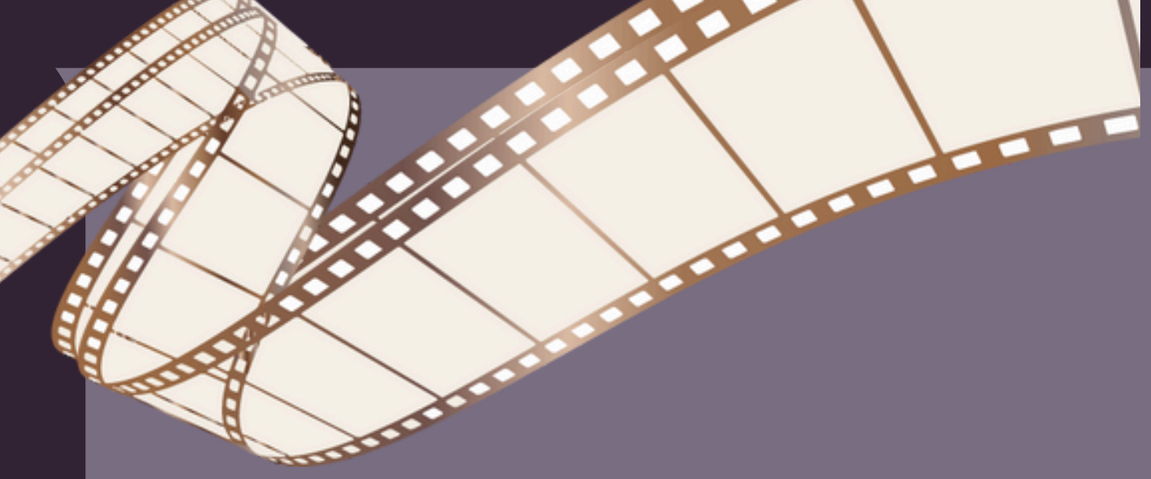
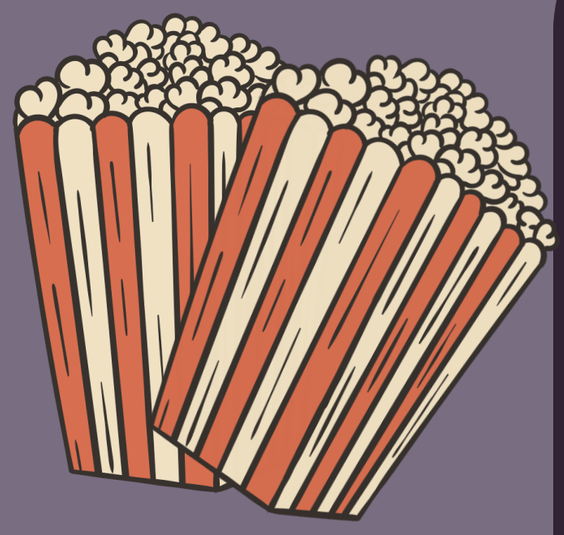




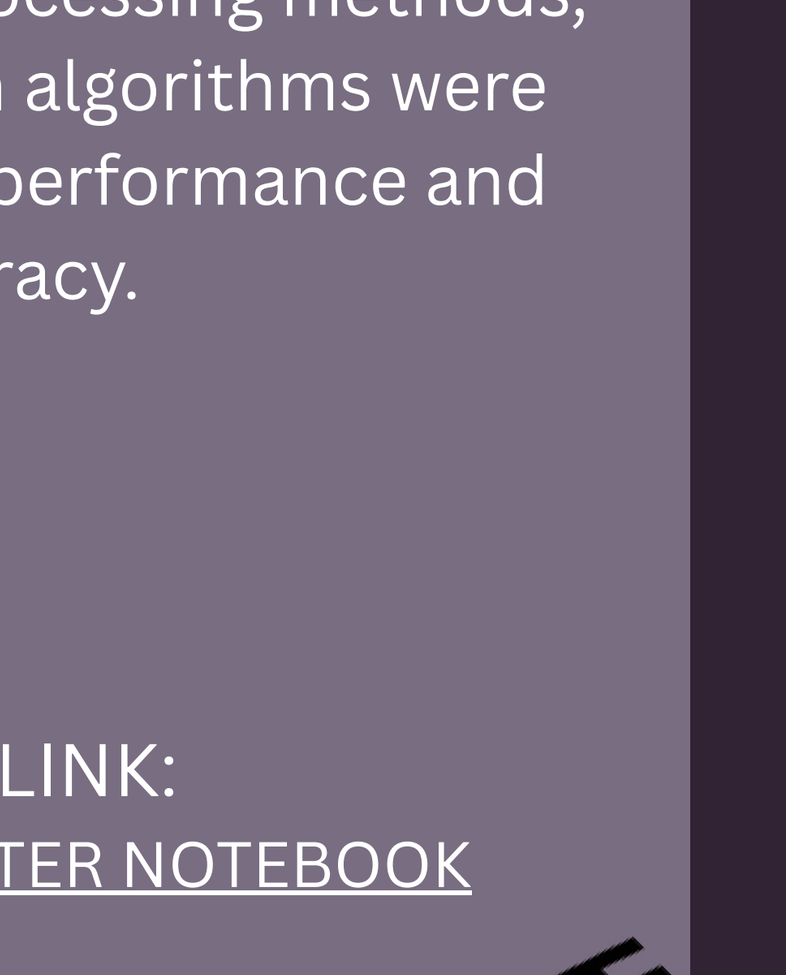
2025-2026

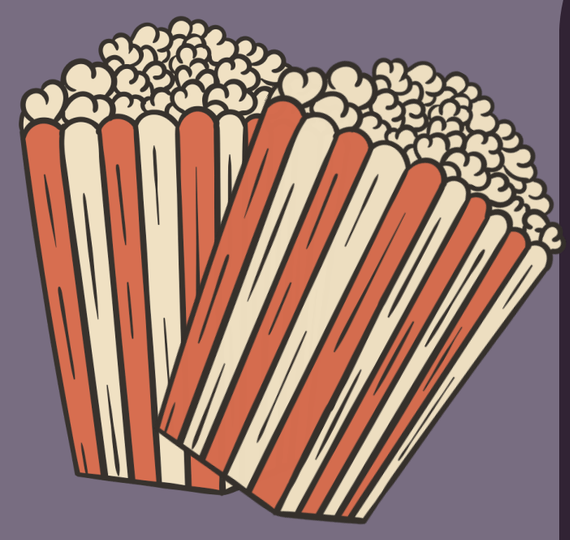
MOVIE SENTIMENT ANALYSIS USING NATURAL LANGUAGE PROCESSING

This project focuses on analyzing movie reviews using Natural Language Processing (NLP) and Machine Learning techniques to classify sentiments as positive or negative. Various preprocessing methods, feature extraction techniques, and classification algorithms were implemented and compared to evaluate model performance and improve sentiment prediction accuracy.

PRESENTED BY:
APOORVA SHARMA

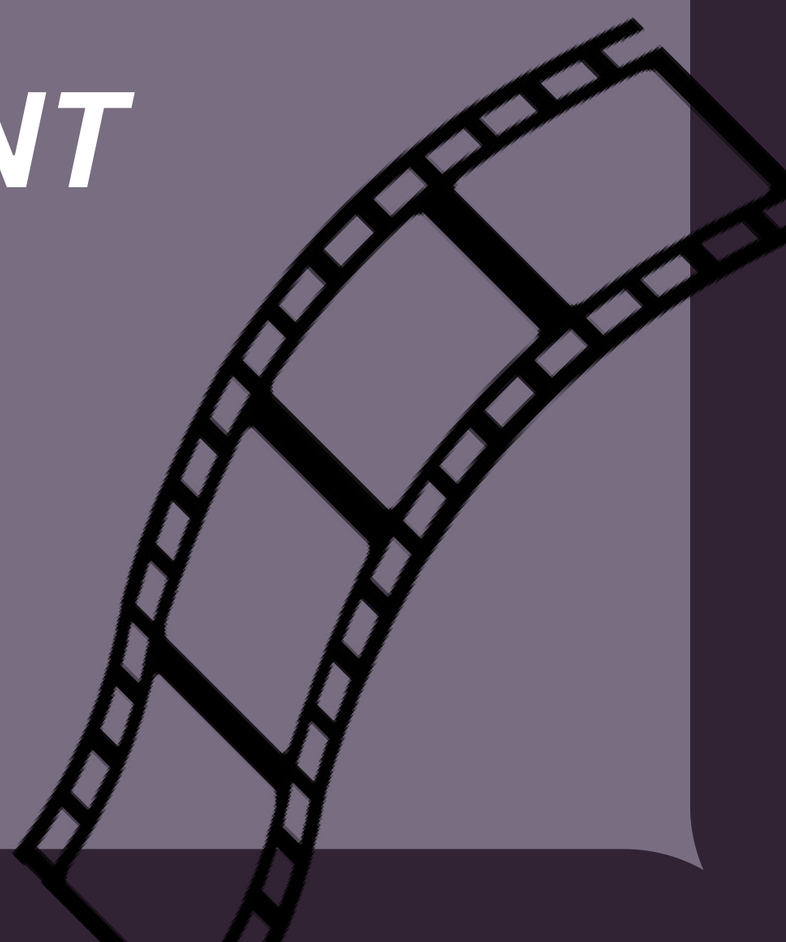
FILE LINK:
JUPYTER NOTEBOOK

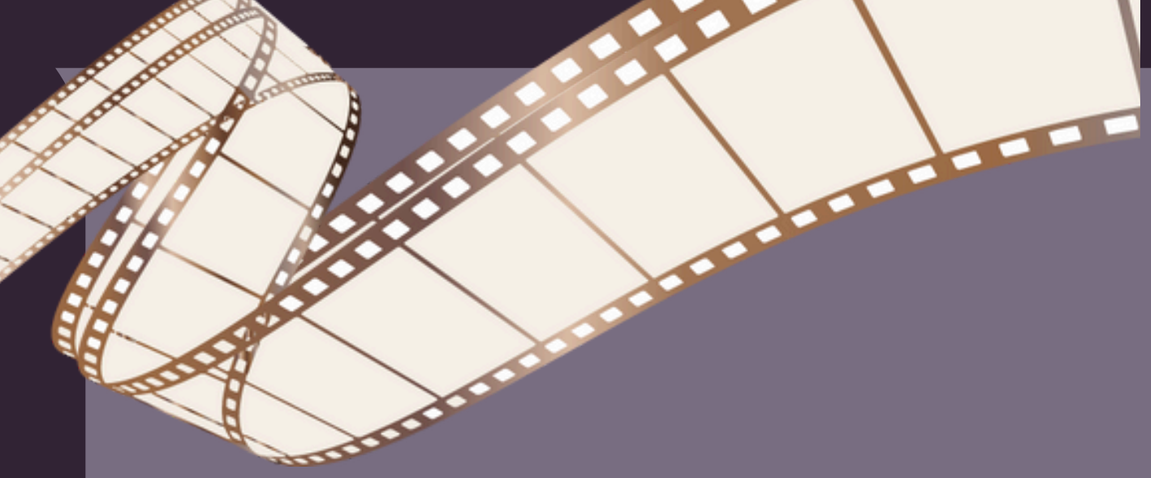


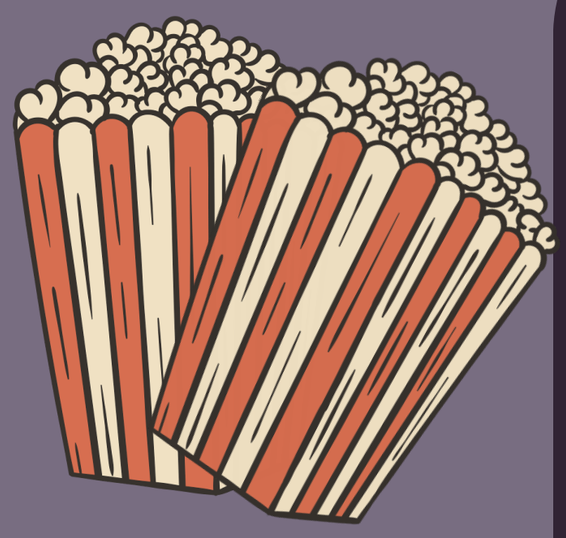


CONTENT

- *Introduction*
- *Objective*
- *Data Overview*
- *Dataset Description*
- *NLP Techniques Used*
- *Algorithms Used*
- *Model Training*
- *Performance Evaluation*
- *Results*
- *Conclusion*
- *Future Scope*
- *ACKNOWLEDGEMENT*







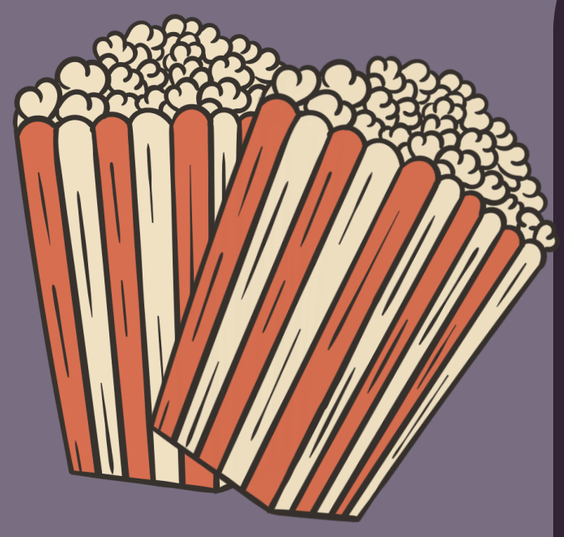
INTRODUCTION

With the rapid growth of digital platforms and online streaming services, users frequently share opinions and reviews about movies on the internet. These reviews contain valuable information that can help understand audience reactions, preferences, and overall sentiment toward a movie. Manually analyzing thousands of reviews is difficult and time-consuming, which creates the need for automated sentiment analysis systems.

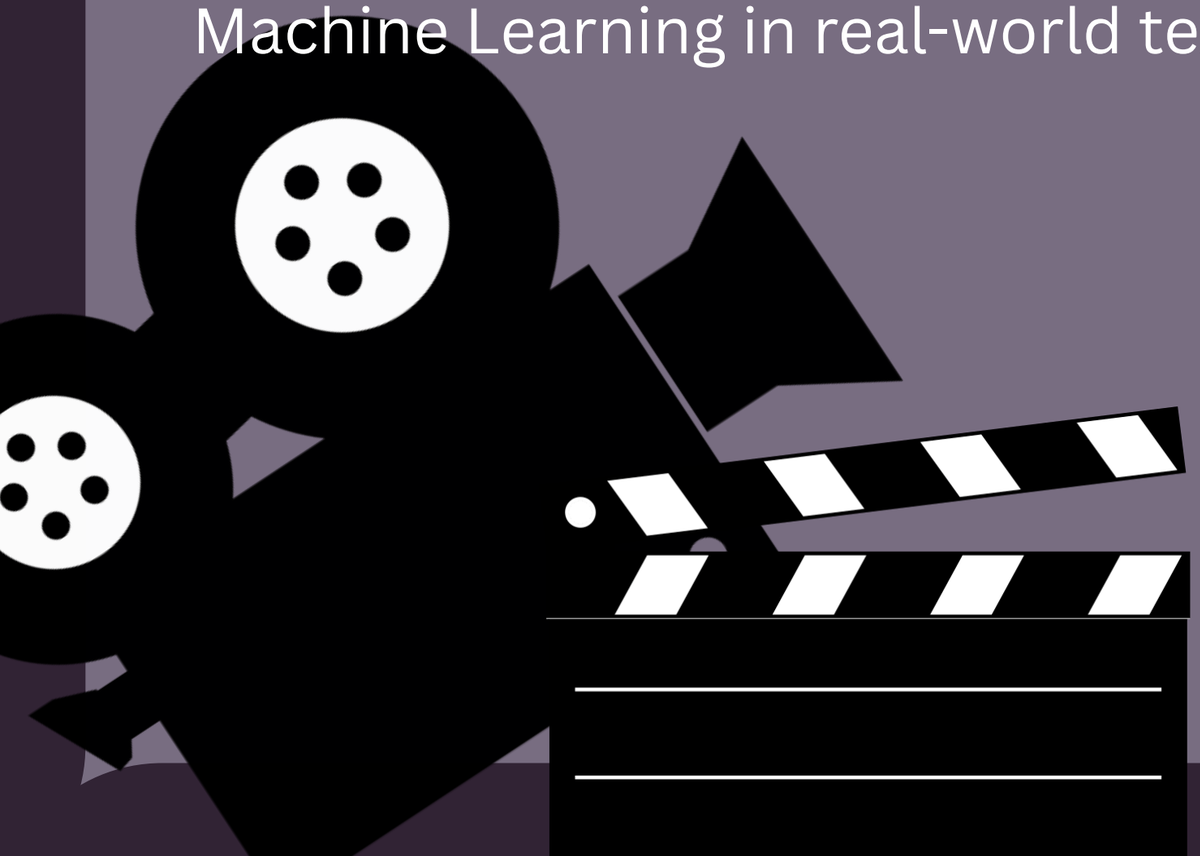
This project focuses on Movie Sentiment Analysis using Natural Language Processing (NLP) and Machine Learning techniques. The main objective is to classify movie reviews as positive or negative based on textual content. Various preprocessing techniques such as tokenization, stopword removal, stemming, and text cleaning were applied to improve data quality. TF-IDF vectorization was used to convert text into numerical form, and multiple machine learning algorithms were implemented and compared to evaluate their performance.

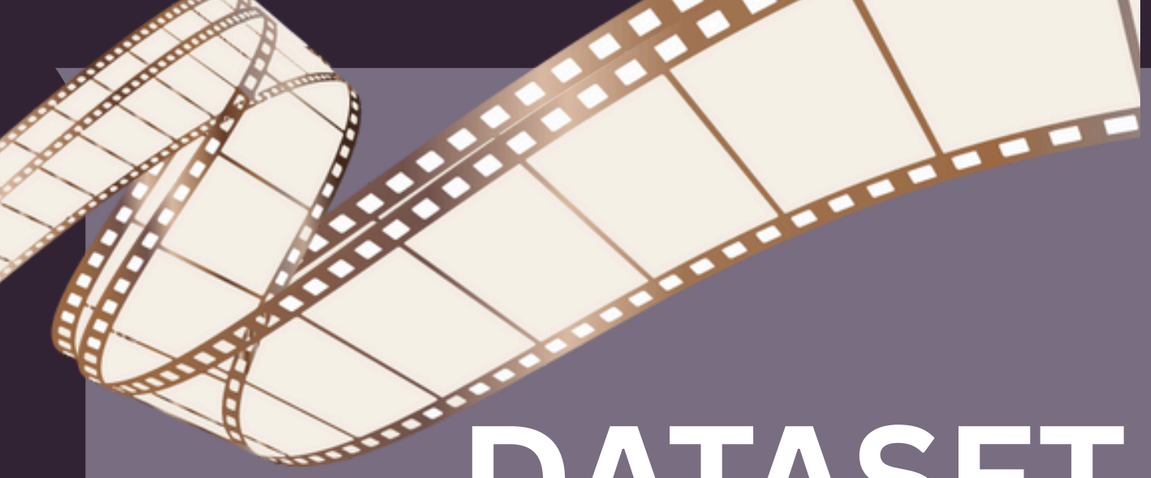
The project demonstrates how NLP can be used to analyze human language and extract meaningful insights from textual data. It also highlights the effectiveness of machine learning models in solving real-world text classification problems.

OBJECTIVE

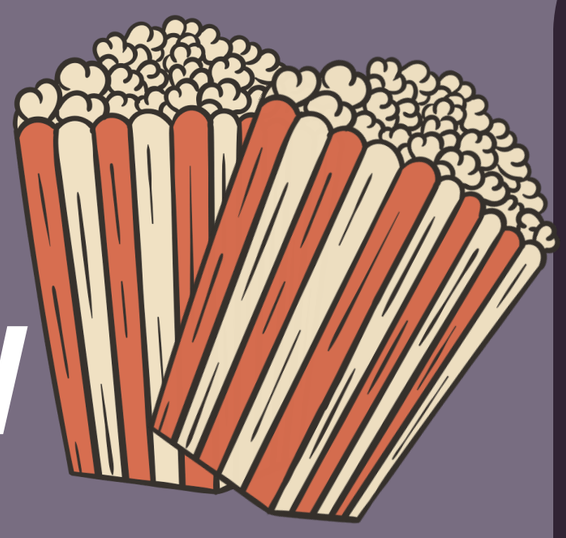


- To collect and analyze movie review datasets for sentiment classification.
- To perform data exploration and understand dataset structure, features, labels, null values, duplicates, and statistical information.
- To clean textual data by removing HTML tags, URLs, punctuation, special characters, and unnecessary spaces.
- To preprocess text using NLP techniques such as lowercasing, tokenization, stopword removal, spelling correction, and stemming.
- To handle slang words and noisy text commonly present in user-generated reviews.
- To convert textual data into numerical form using feature extraction techniques like Bag of Words (BOW) and TF-IDF.
- To encode categorical sentiment labels using encoding techniques such as Label Encoding and One-Hot Encoding.
- To split the dataset into training and testing data for model development.
- To implement and compare multiple machine learning algorithms including Logistic Regression, Naive Bayes, Decision Tree, Random Forest, SVM, and XGBoost.
- To evaluate model performance using metrics such as Accuracy Score, Classification Report, and Confusion Matrix.
- To visualize and compare the performance of different machine learning models using graphs and charts.
- To build an efficient sentiment prediction system capable of classifying movie reviews as positive or negative.
- To understand the practical applications of Natural Language Processing and Machine Learning in real-world text classification problems.





DATASET OVERVIEW

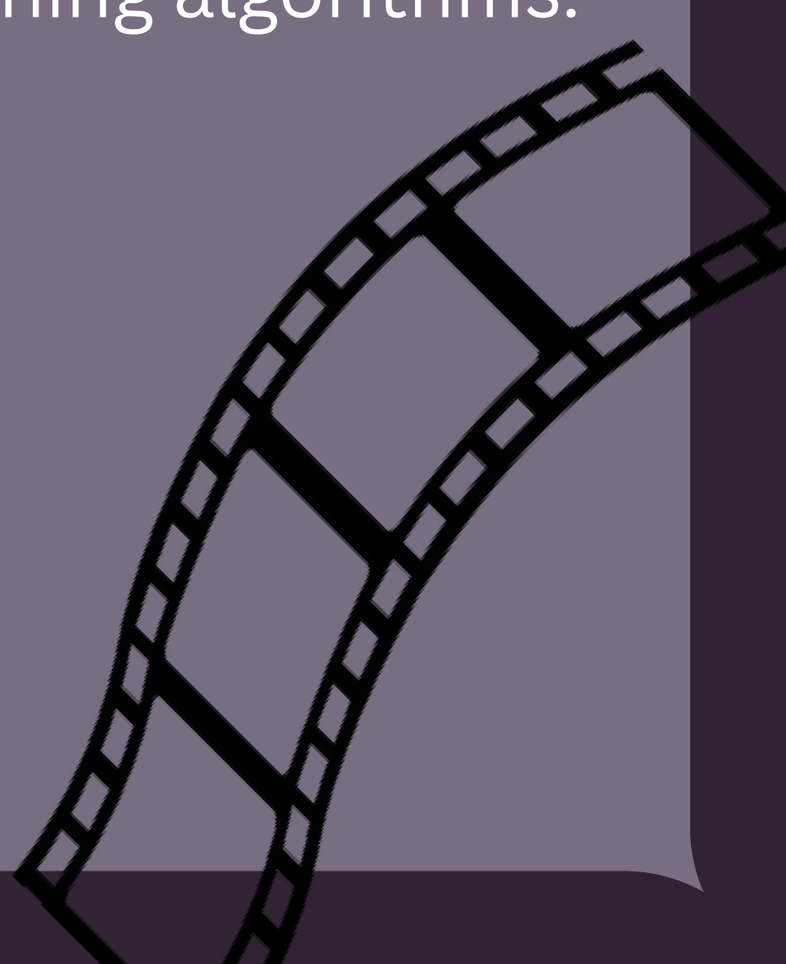


The dataset used in this project is the IMDb Movie Reviews Dataset, which contains a large collection of movie reviews along with their corresponding sentiment labels. The dataset is widely used for Natural Language Processing (NLP) and sentiment analysis tasks.

The dataset consists of 50,000 movie reviews divided into two columns: review and sentiment. The review column contains textual movie reviews provided by users, while the sentiment column contains the sentiment label associated with each review, categorized as either positive or negative.

The dataset includes both favorable and unfavorable reviews, making it suitable for binary text classification problems. Since the data is textual in nature, several preprocessing techniques such as HTML tag removal, tokenization, stopword removal, stemming, and text vectorization were applied before training machine learning models.

The balanced distribution of positive and negative reviews helps in building an unbiased sentiment classification system and provides reliable evaluation of machine learning algorithms.



DATA DESCRIPTION

```
df.info()
```

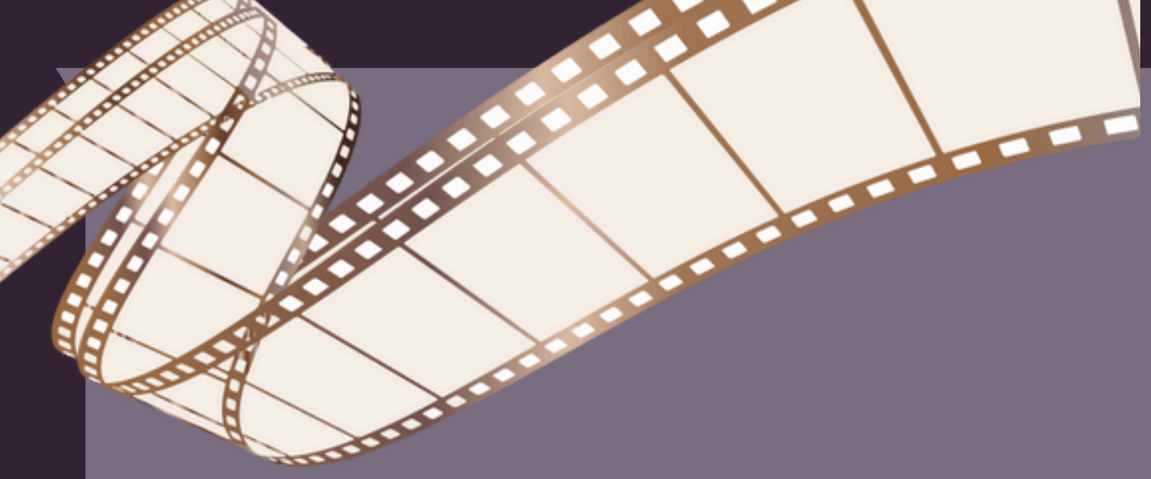
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 2 columns):
 #   Column     Non-Null Count  Dtype 
---  ------     --------------  ----- 
 0   review     50000 non-null  object
 1   sentiment  50000 non-null  object
dtypes: object(2)
memory usage: 781.4+ KB
```

The dataset consists of 50,000 entries with 2 columns namely review and sentiment. Both columns contain textual data with the object data type. The review column stores movie review text, while the sentiment column contains the corresponding sentiment labels such as positive or negative. The dataset contains no null values, indicating that all records are complete and suitable for preprocessing and analysis. With a memory usage of approximately 781.4 KB, the dataset is well-structured and appropriate for performing Natural Language Processing (NLP) and sentiment classification tasks.

```
df.describe()
```

	review	sentiment
count	50000	50000
unique	49582	2
top	Loved today's show!!! It was a variety and not...	positive
freq	5	25000

The descriptive analysis of the dataset shows that it contains 50,000 movie reviews and corresponding sentiment labels. Out of these, 49,582 reviews are unique, indicating the presence of a few duplicate reviews in the dataset. The sentiment column contains only two unique categories: positive and negative, making it a binary classification problem. The most frequently occurring sentiment is positive with a frequency of 25,000, showing that the dataset is balanced with an equal distribution of positive and negative reviews. This balanced nature of the dataset helps in building reliable and unbiased machine learning models for sentiment analysis.



```
df.shape
```

```
(50000, 2)
```

```
df.columns
```

```
Index(['review', 'sentiment'], dtype='object')
```

```
df.duplicated().sum()
```

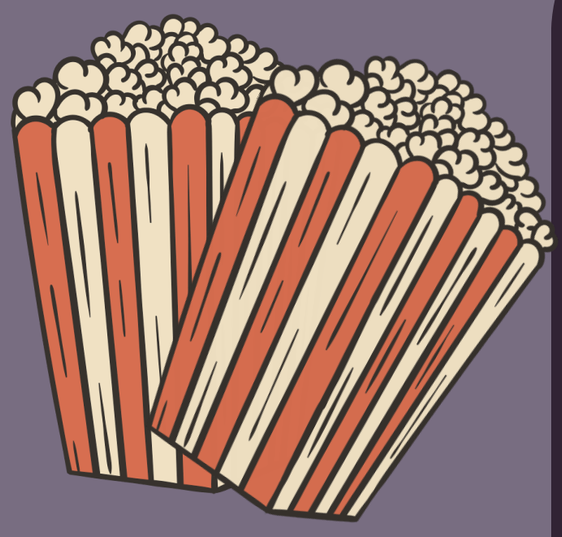
```
np.int64(418)
```

```
df = df.drop_duplicates()
```

```
df.isnull().sum()
```

```
review       0
sentiment    0
dtype: int64
```

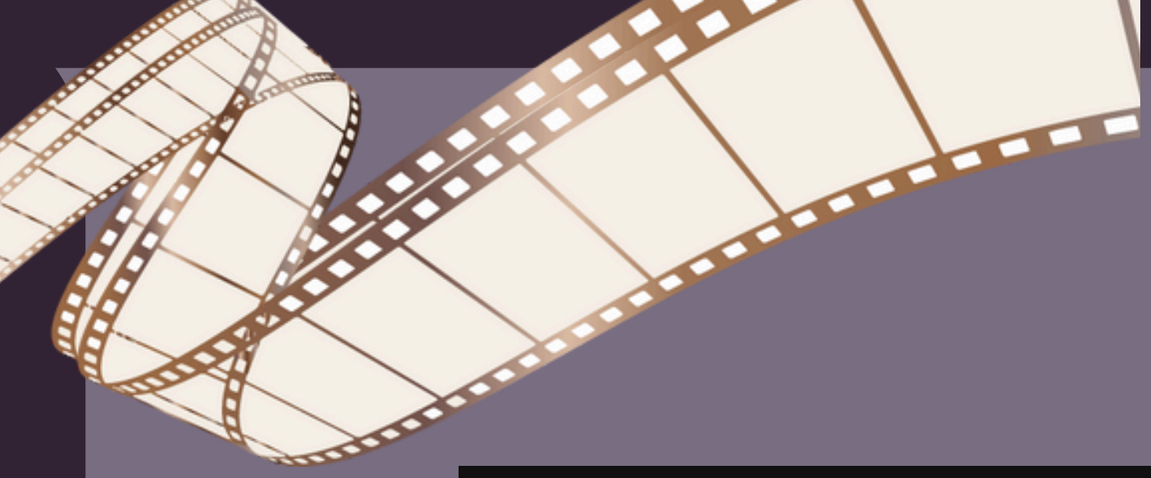
The dataset exploration and preprocessing stage was performed using functions such as df.columns, df.shape, df.duplicated(), drop_duplicates(), and isnull().sum() to understand and clean the movie review dataset effectively. The dataset contained two main columns, namely review and sentiment, used for sentiment classification. The df.shape function helped identify the size and structure of the dataset, while df.columns displayed the available features. During preprocessing, 418 duplicate records were identified using df.duplicated() and removed using drop_duplicates() to improve data quality and avoid redundancy. Additionally, isnull().sum() confirmed that the dataset contained no missing or null values, indicating that the data was complete and suitable for further NLP preprocessing and machine learning model training.

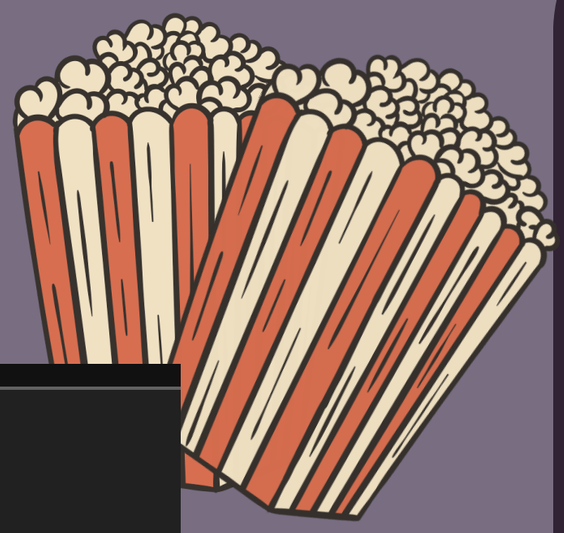


NLP TECHNIQUES USED

```
df['review'] = df['review'].str.lower()
```

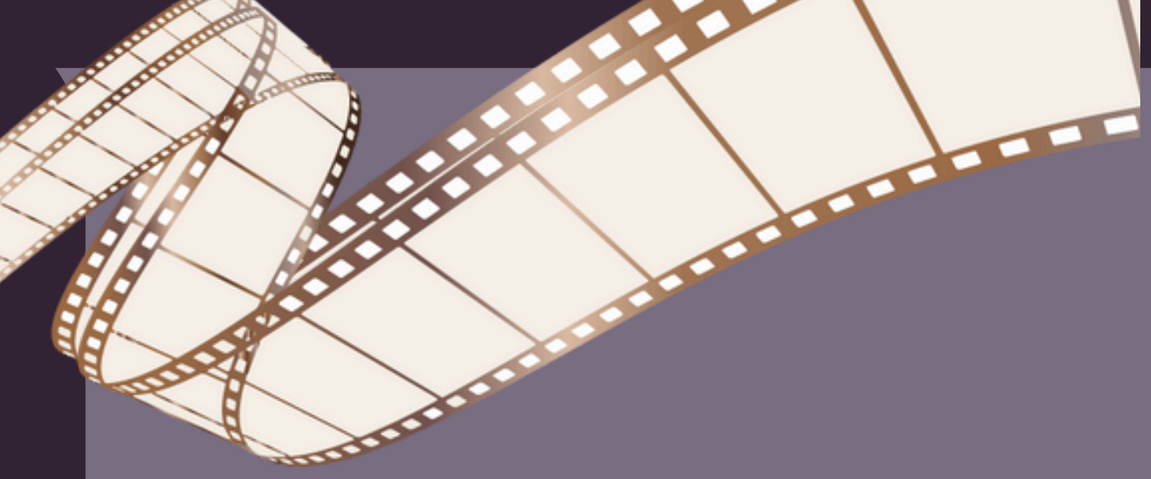
The Natural Language Processing (NLP) preprocessing stage began with the use of the str.lower() function to convert all movie reviews into lowercase text format. This step helped maintain consistency in the textual data by treating words such as “Good”, “GOOD”, and “good” as the same word during analysis. Lowercasing reduced unnecessary variations in the dataset and improved the effectiveness of further preprocessing techniques such as tokenization, stopword removal, stemming, and TF-IDF feature extraction, ultimately helping improve the performance of the sentiment classification models.

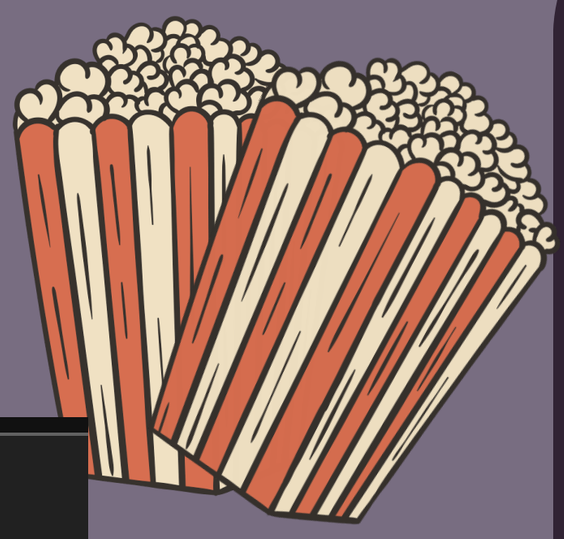




```python
import re
def remove_html_tags(text):
    if not isinstance(text, str):
        return text
    pattern = re.compile(r'<.*?>')
    return pattern.sub('', text)


df['review'] = df['review'].apply(remove_html_tags)
```

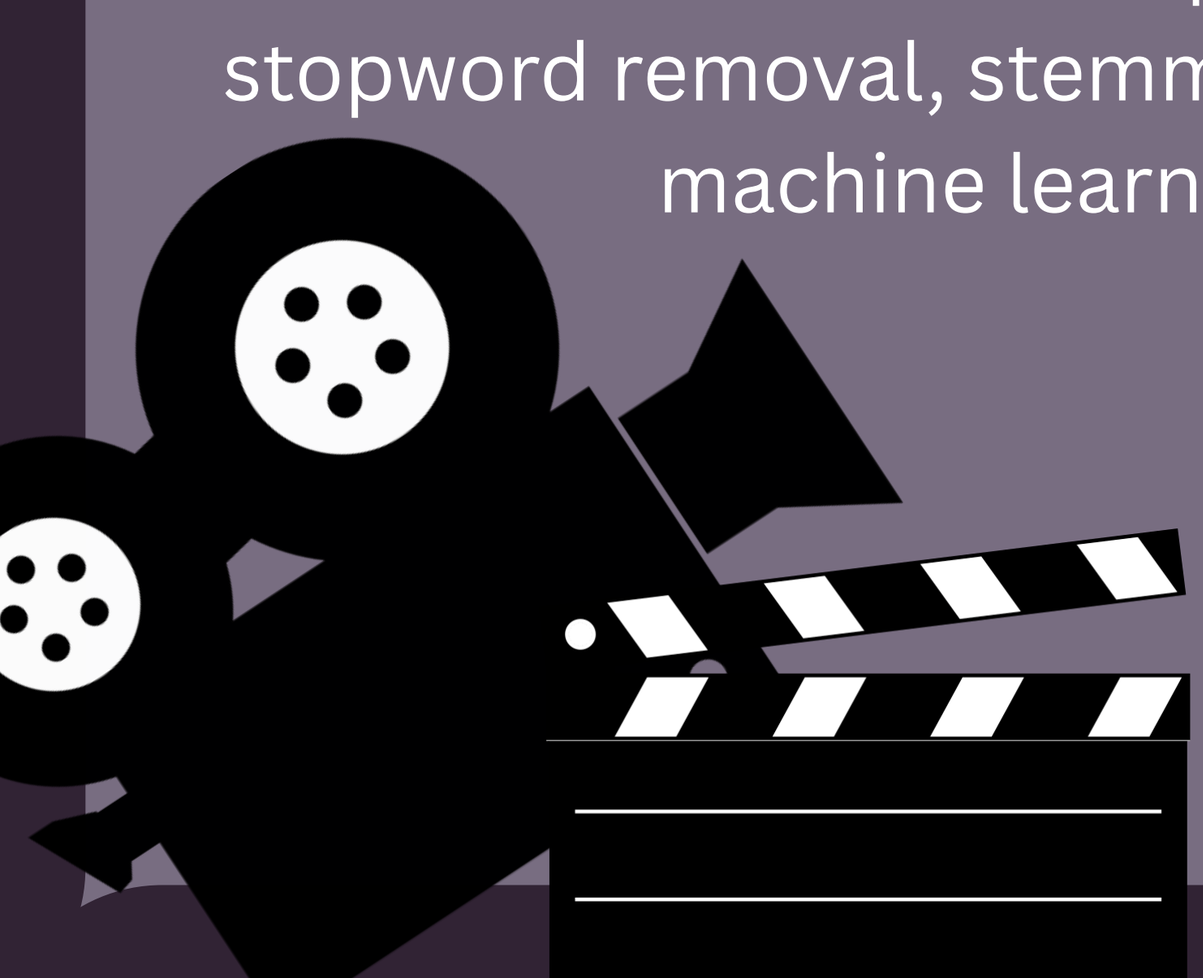
HTML tags were removed from the movie review dataset using Regular Expressions (Regex) as part of the text cleaning and preprocessing stage in Natural Language Processing. A custom function named remove_html_tags() was created using Python’s re library, where the regex pattern r'<.*?>' was used to detect and eliminate HTML tags such as
, <p>, and other formatting elements present in the review text. The function was then applied to the review column using the apply() method to clean every movie review individually. This preprocessing step helped transform the raw reviews into clean and readable plain text by removing unnecessary web-based formatting content that does not contribute to sentiment prediction. Removing HTML tags improved the overall text quality and enhanced the efficiency of further NLP operations such as tokenization, stopword removal, stemming, TF-IDF vectorization, and machine learning model training.

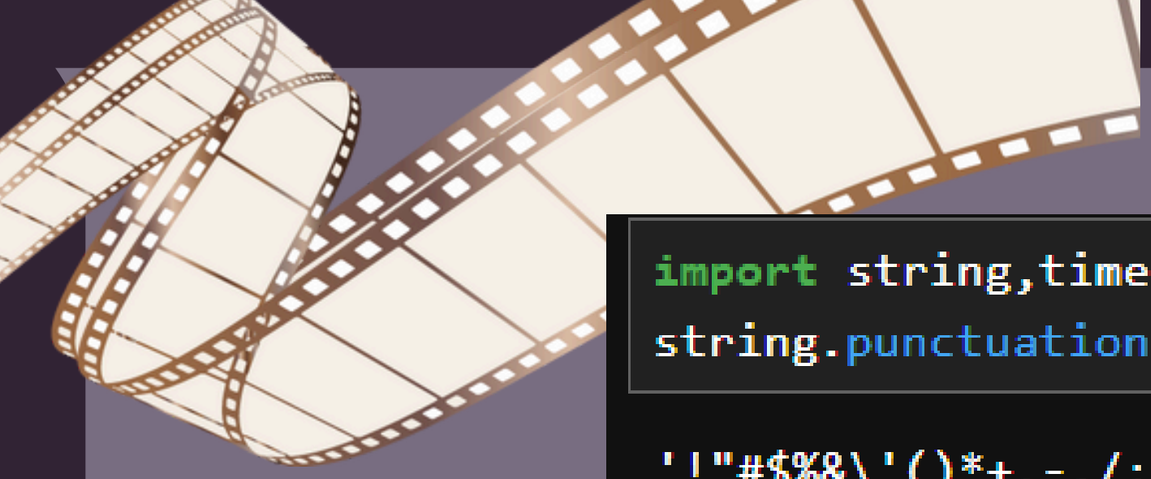


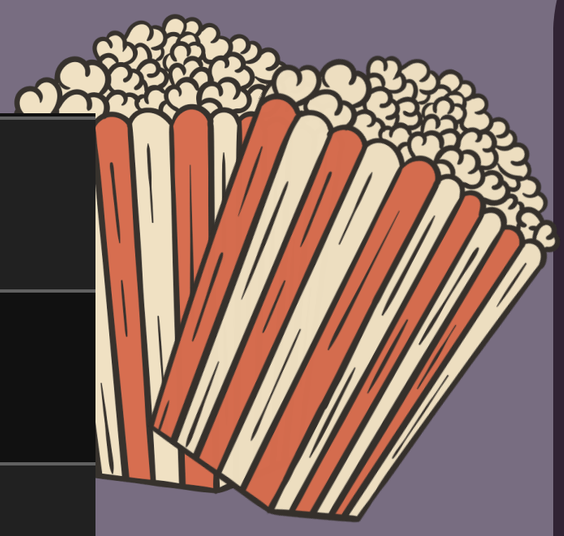


```python
def remove_url(text):
    if not isinstance(text, str):
        return text
    pattern = re.compile(r'https?://\S+|www\.\S+')
    return pattern.sub(r'', text)
df['review'] = df['review'].apply(remove_url)
```

URLs and website links were removed from the movie review dataset using Regular Expressions (Regex) as part of the text preprocessing and cleaning stage in Natural Language Processing. A custom function named remove_url() was created using Python's re library, where the regex pattern r'https?://\S+|www\.\S+' was used to identify and eliminate URLs beginning with http, https, or www from the review text. The function was then applied to the review column using the apply() method to clean every movie review individually. Removing URLs helped eliminate unnecessary external links and noisy textual information that do not contribute to sentiment prediction. This preprocessing step improved the quality and consistency of the textual data, making it more suitable for further NLP operations such as tokenization, stopword removal, stemming, TF-IDF vectorization, and machine learning model training.







```
import string,time
string.punctuation
```

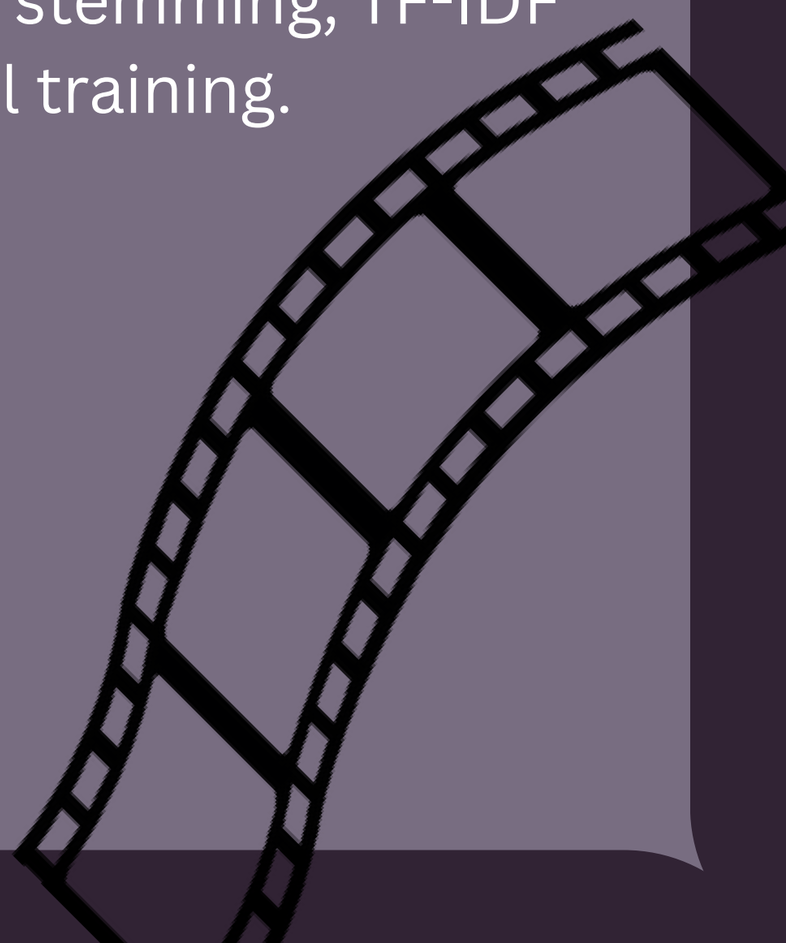
```
'!"#$%&\'()*+,-./:;<=>?@[\\]^_`{|}~'
```

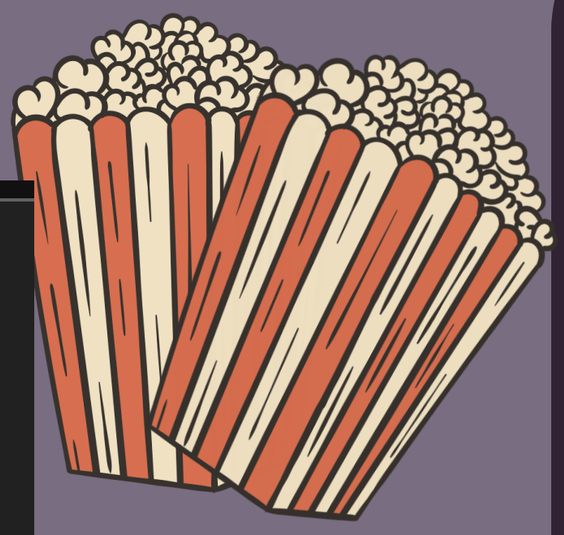
```
exclude = string.punctuation
```

```
def remove_punc1(text):
    if not isinstance(text, str):
        return text
    for char in exclude:
        text = text.replace(char,'')
    return text
```

```
df['review'] = df['review'].apply(remove_punc1)
```

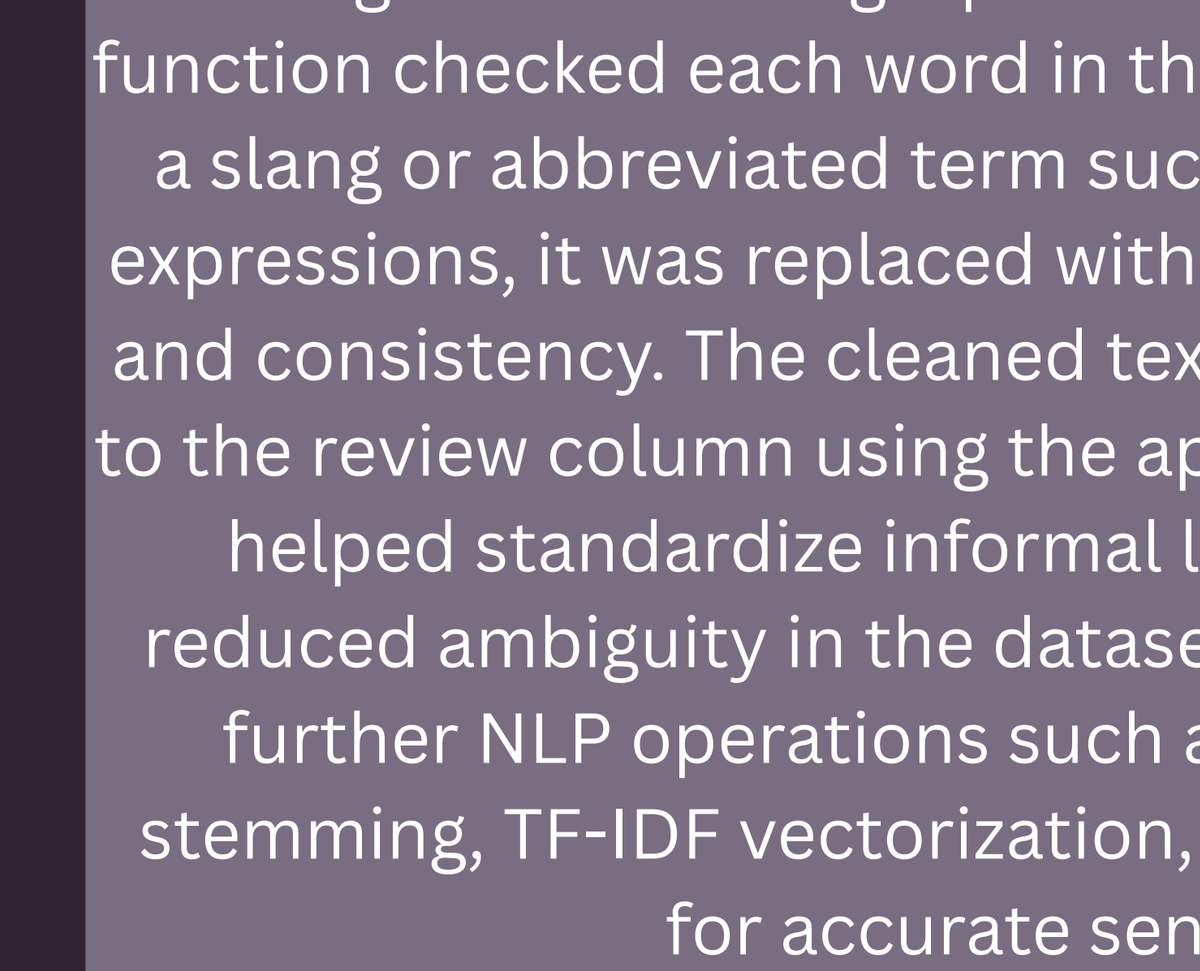
Punctuation marks were removed from the movie review dataset as part of the text cleaning and preprocessing stage in Natural Language Processing. The string.punctuation function from Python’s string library was used to identify all punctuation symbols such as commas, periods, question marks, exclamation marks, brackets, and special characters present in the review text. A custom function named remove_punc1() was created to iterate through each punctuation character and replace it with an empty string, effectively cleaning the textual data. The function was then applied to the review column using the apply() method to preprocess every movie review individually. Removing punctuation helped reduce noise in the dataset and ensured that unnecessary symbols did not affect tokenization, feature extraction, and sentiment prediction. This preprocessing step improved the consistency and quality of the textual data, making it more suitable for further NLP operations such as stopword removal, stemming, TF-IDF vectorization, and machine learning model training.

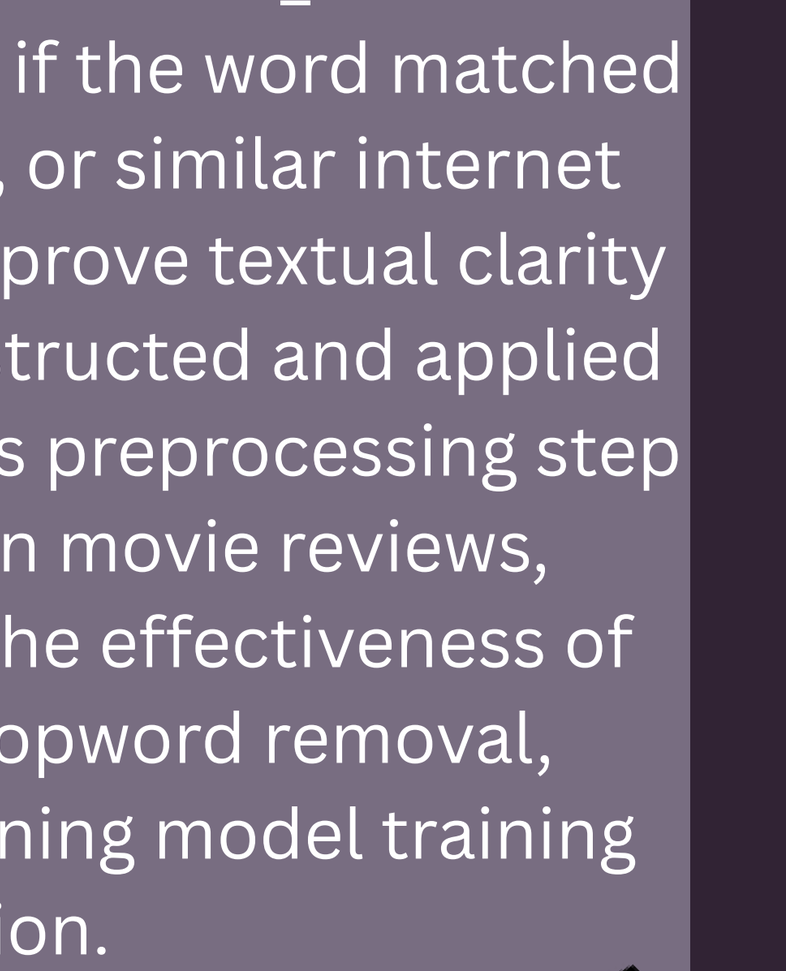


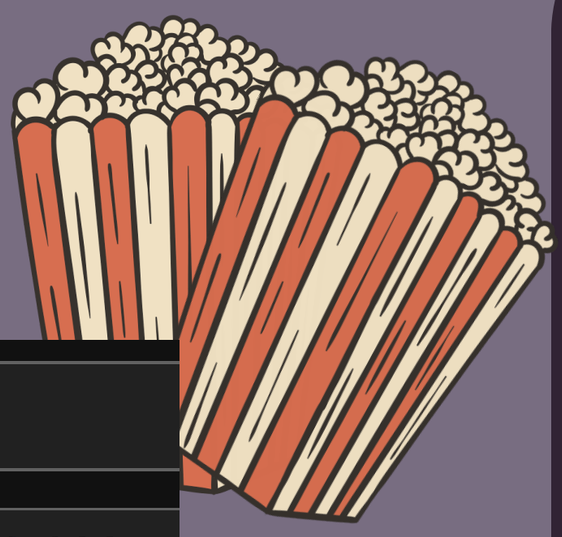


```python
def chat_conversion(text):
    if not isinstance(text, str):
        return text
    new_text = []
    for w in text.split():
        if w.upper() in chat_words:
            new_text.append(chat_words[w.upper()])
        else:
            new_text.append(w)
    return " ".join(new_text)
df['review'] = df['review'].apply(chat_conversion)
```

Chat word conversion was performed as part of the Natural Language Processing preprocessing stage to handle slang words, abbreviations, and informal text commonly found in user-generated movie reviews. A custom function named chat_conversion() was created to identify abbreviated chat words and replace them with their complete meaningful forms using a predefined dictionary called chat_words. The function checked each word in the review text, and if the word matched a slang or abbreviated term such as “LOL”, “OMG”, or similar internet expressions, it was replaced with its full form to improve textual clarity and consistency. The cleaned text was then reconstructed and applied to the review column using the apply() method. This preprocessing step helped standardize informal language present in movie reviews, reduced ambiguity in the dataset, and improved the effectiveness of further NLP operations such as tokenization, stopword removal, stemming, TF-IDF vectorization, and machine learning model training for accurate sentiment classification.



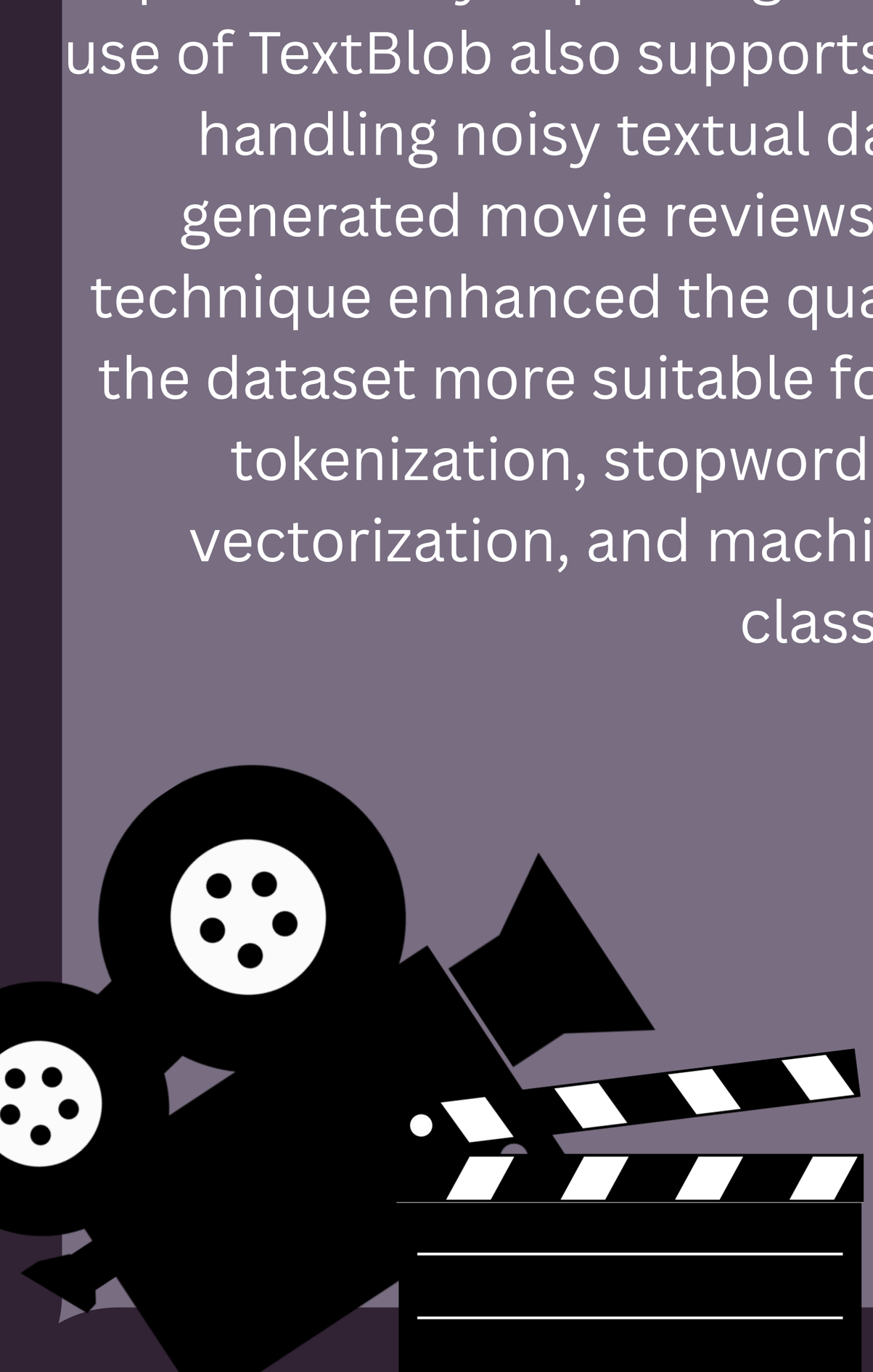


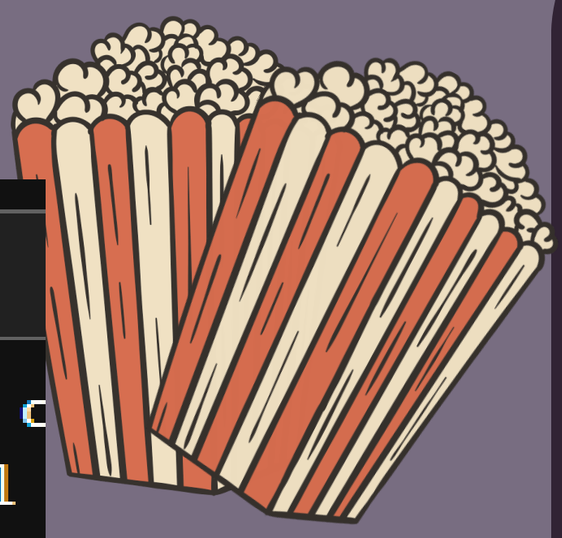


```
from textblob import TextBlob
```

```
df['review'] = df['review'].apply(lambda x: str(TextBlob(x)))
```

The TextBlob library was used during the NLP preprocessing stage to further standardize and process the textual movie review data. The TextBlob() function was applied to each review in the dataset using the apply() method, converting the review text into a structured text object and then back into string format using str(). This step helped in preparing the textual data for advanced Natural Language Processing operations by improving text handling and consistency. The use of TextBlob also supports text normalization and assists in handling noisy textual data commonly found in user-generated movie reviews. Applying this preprocessing technique enhanced the quality of the review text and made the dataset more suitable for subsequent NLP tasks such as tokenization, stopword removal, stemming, TF-IDF vectorization, and machine learning-based sentiment classification.





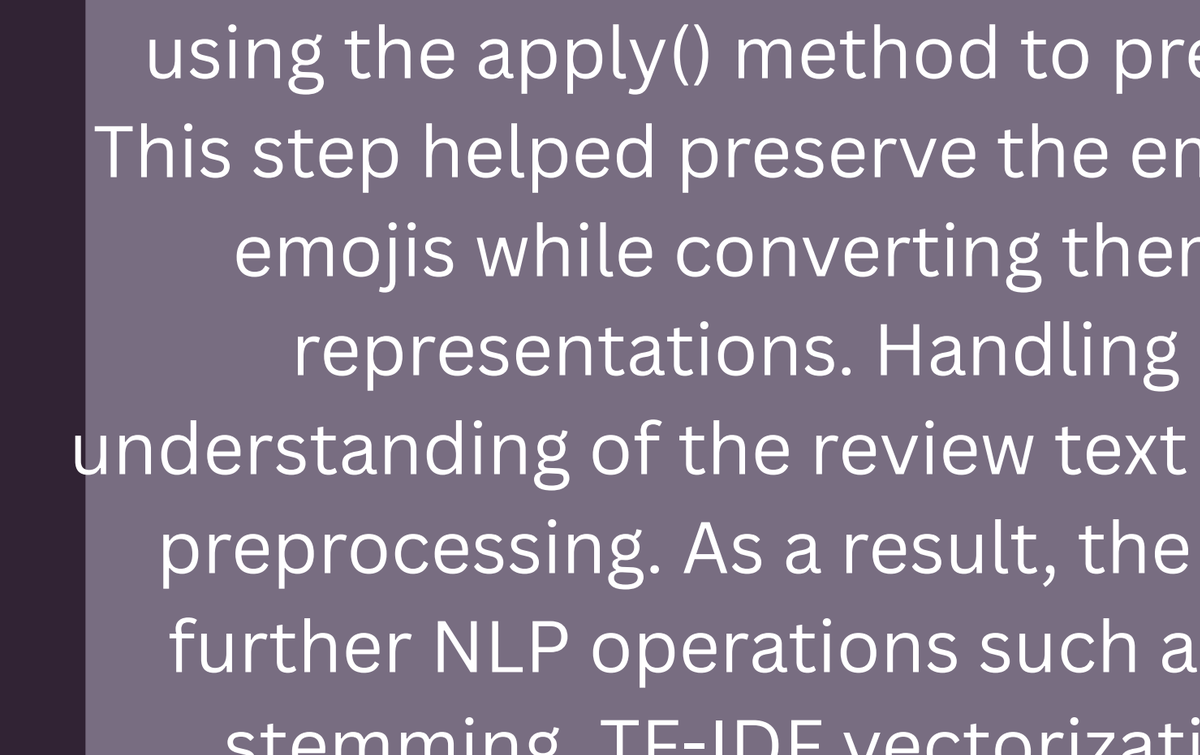
```
pip install emoji
```

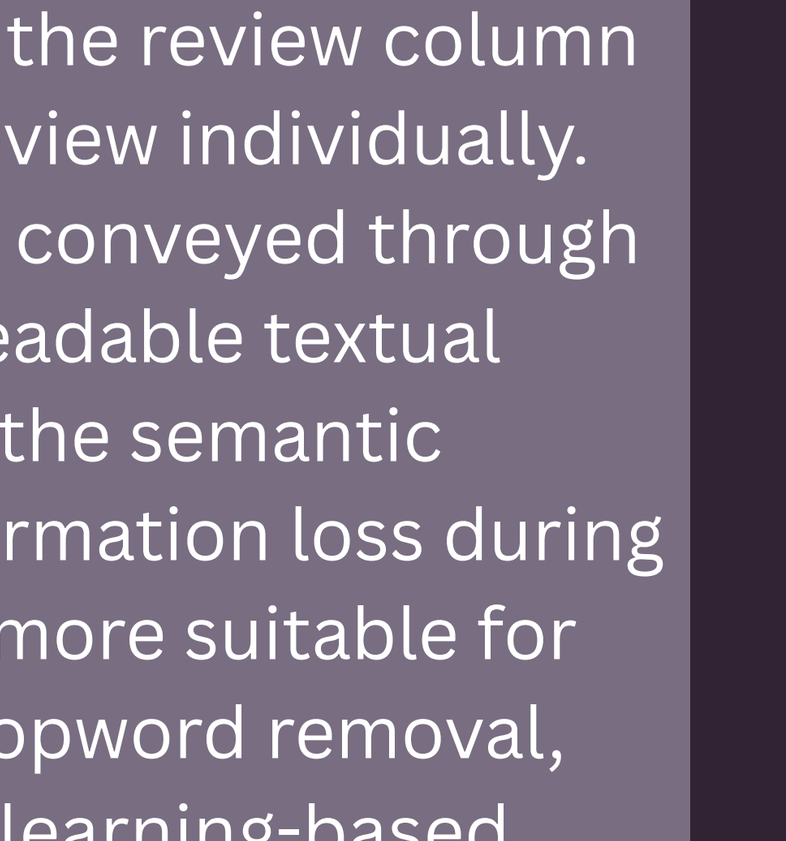
```
Requirement already satisfied: emoji in c
Note: you may need to restart the kernel
```

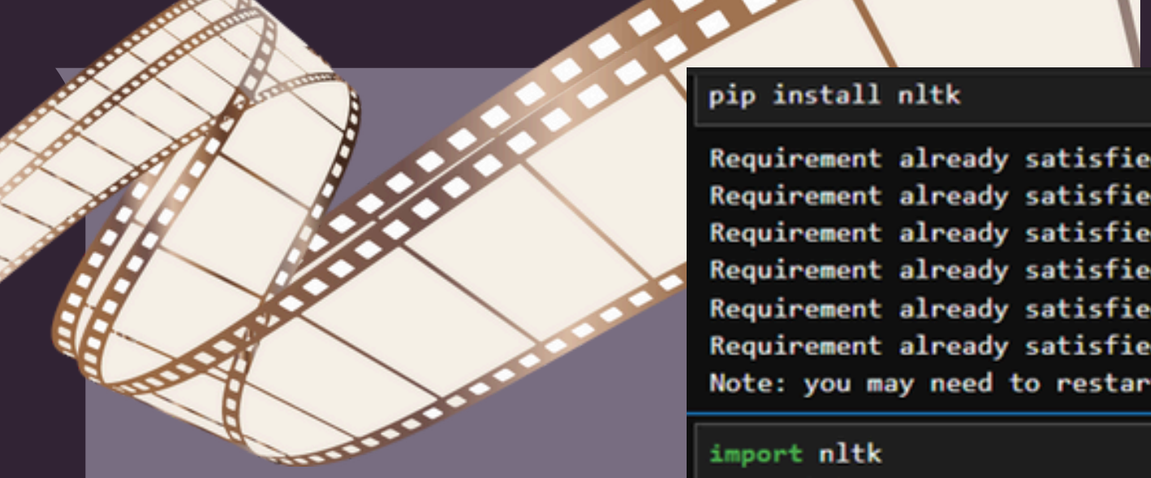
```
import emoji
def em(text):
  return emoji.demojize(text)
```

```
df['review'] = df['review'].apply(em)
```

Emoji conversion was performed during the NLP preprocessing stage using the emoji library to handle emojis present in user-generated movie reviews. The emoji.demojize() function was used inside a custom function named em() to convert emojis into their corresponding textual meanings. For example, emojis such as 😊 or 😢 were transformed into descriptive text formats like :smiling_face: or :crying_face:. The function was then applied to the review column using the apply() method to preprocess every review individually. This step helped preserve the emotional meaning conveyed through emojis while converting them into machine-readable textual representations. Handling emojis improved the semantic understanding of the review text and reduced information loss during preprocessing. As a result, the dataset became more suitable for further NLP operations such as tokenization, stopword removal, stemming, TF-IDF vectorization, and machine learning-based sentiment classification.







```
pip install nltk
```

```
Requirement already satisfied: nltk in c:\users\apoorva\anaconda3\lib\site-
Requirement already satisfied: click in c:\users\apoorva\anaconda3\lib\site
Requirement already satisfied: joblib in c:\users\apoorva\anaconda3\lib\sit
Requirement already satisfied: regex>=2021.8.3 in c:\users\apoorva\anaconda
Requirement already satisfied: tqdm in c:\users\apoorva\anaconda3\lib\site-
Requirement already satisfied: colorama in c:\users\apoorva\anaconda3\lib\s
Note: you may need to restart the kernel to use updated packages.
```

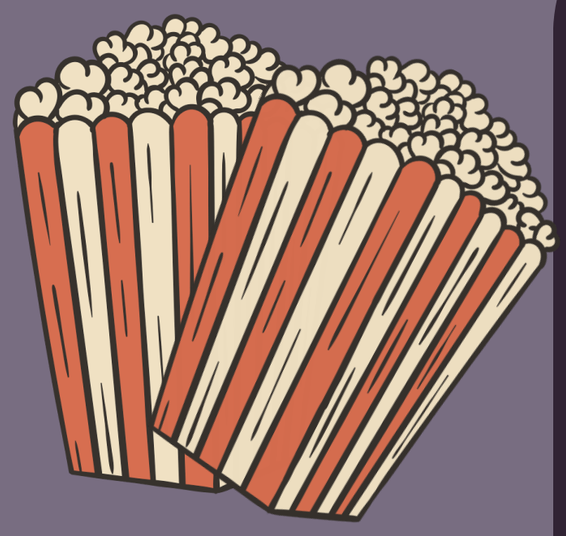
```
import nltk
from nltk.corpus import stopwords

nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\Apoorva\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

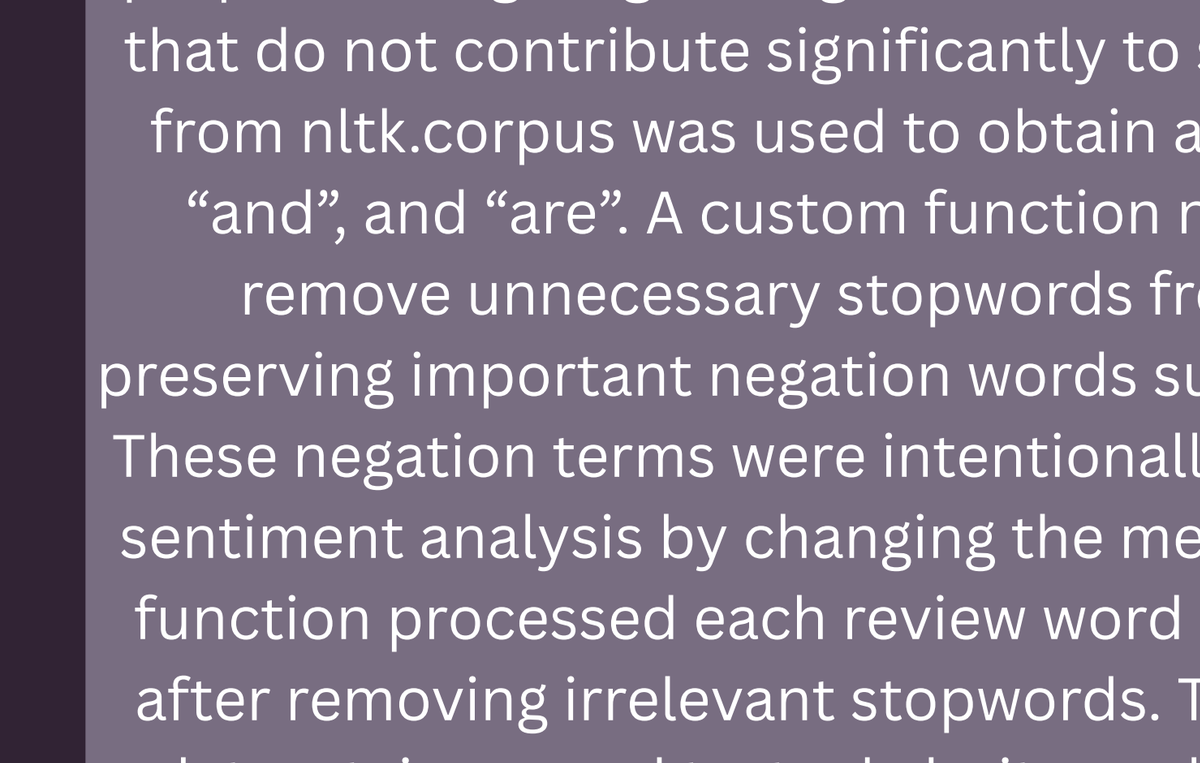
```
True
```

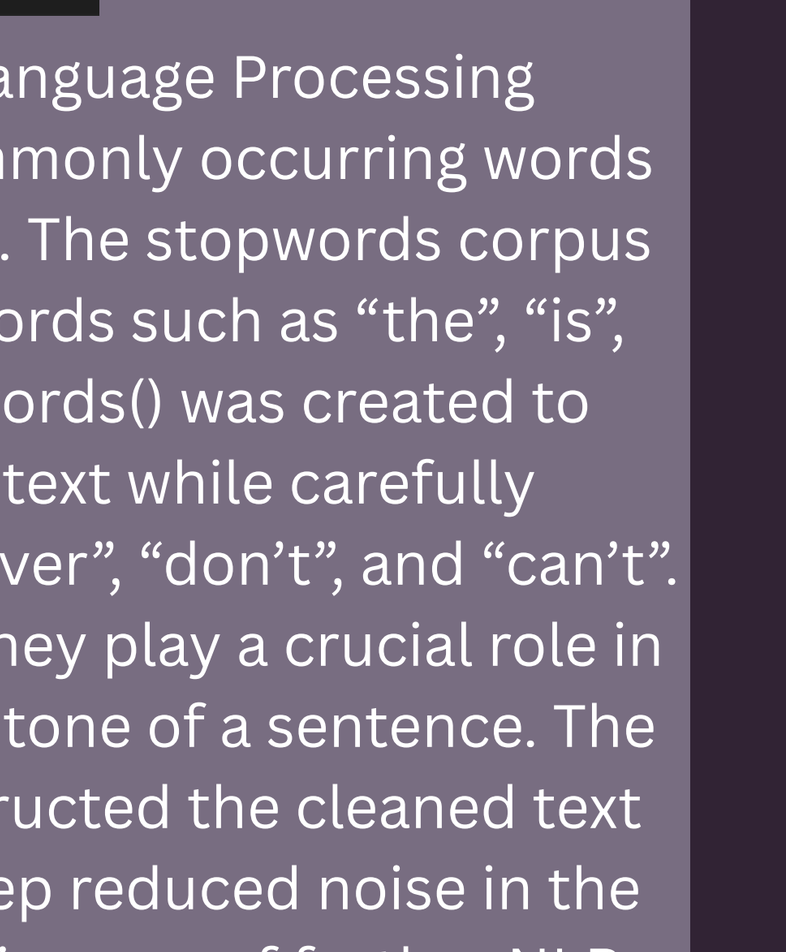
```
stopword = stopwords.words('english')
```

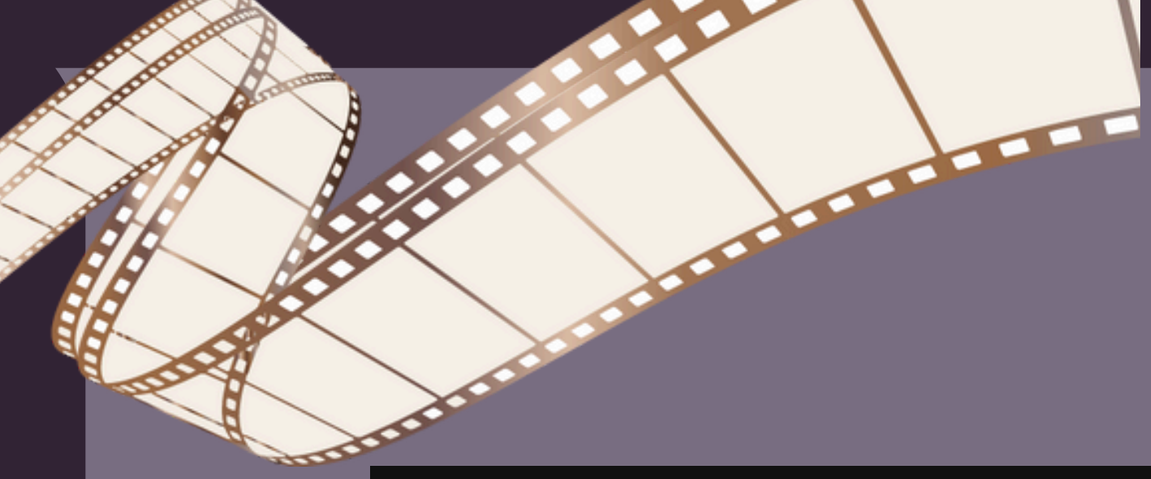
```
def remove_stopwords(text):
    new_text = []
    sw = set(stopwords.words('english'))
    negations_to_keep = {
        "not", "no", "nor", "never", "none",
        "n't", "don't", "doesn't", "didn't", "won't", "wouldn't",
        "shouldn't", "couldn't", "can't", "cannot", "ain't"
    }

    for word in text.split():
        w = word.lower()
        if w in negations_to_keep or w.endswith(("n't", "not")):
            new_text.append(word)
        elif w in sw:
            continue

        else:
            new_text.append(word)

    return " ".join(new_text)
```

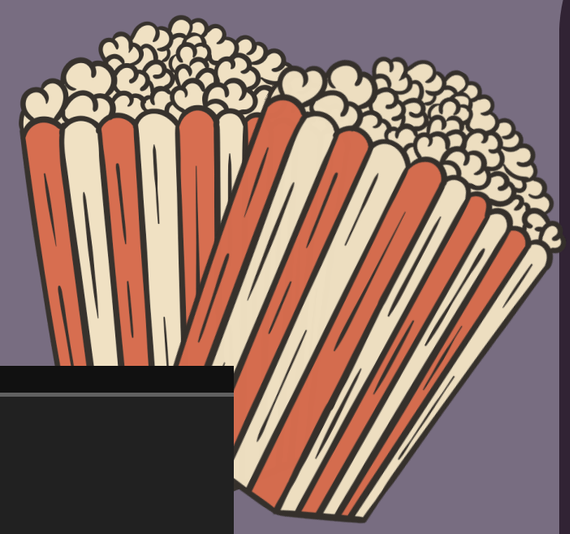


Stopword removal was performed during the Natural Language Processing preprocessing stage using the NLTK library to eliminate commonly occurring words that do not contribute significantly to sentiment prediction. The stopwords corpus from nltk.corpus was used to obtain a list of English stopwords such as “the”, “is”, “and”, and “are”. A custom function named remove_stopwords() was created to remove unnecessary stopwords from the movie review text while carefully preserving important negation words such as “not”, “no”, “never”, “don’t”, and “can’t”. These negation terms were intentionally retained because they play a crucial role in sentiment analysis by changing the meaning and emotional tone of a sentence. The function processed each review word by word and reconstructed the cleaned text after removing irrelevant stopwords. This preprocessing step reduced noise in the dataset, improved textual clarity, and enhanced the effectiveness of further NLP operations such as tokenization, stemming, TF-IDF vectorization, and machine learning model training for accurate movie review sentiment classification.





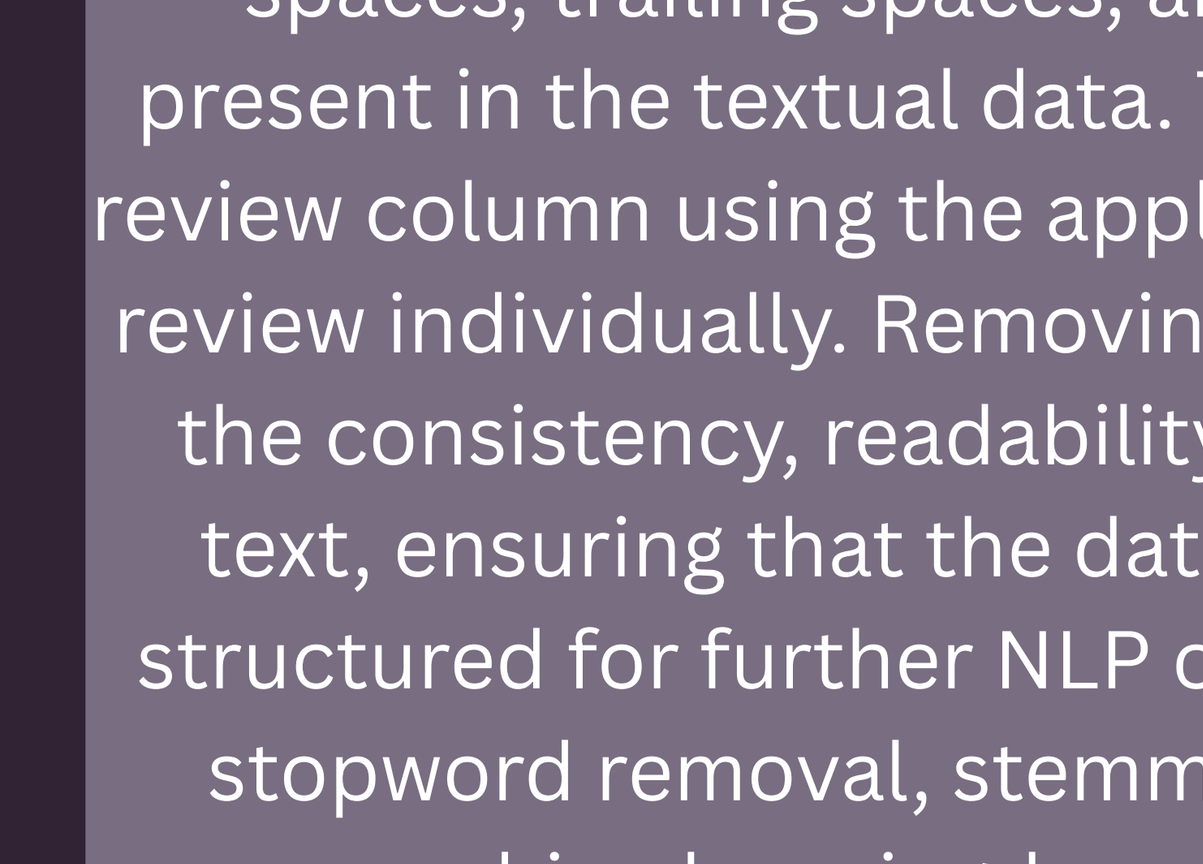


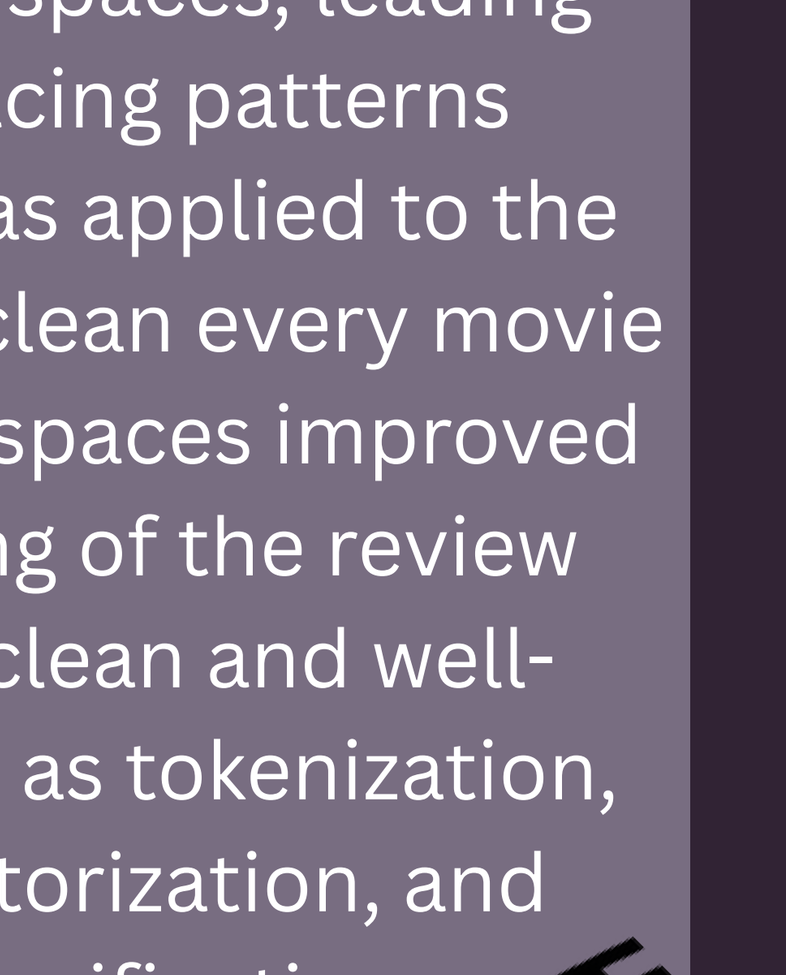


```python
def remove_spaces(text):
    return " ".join(text.split())
```

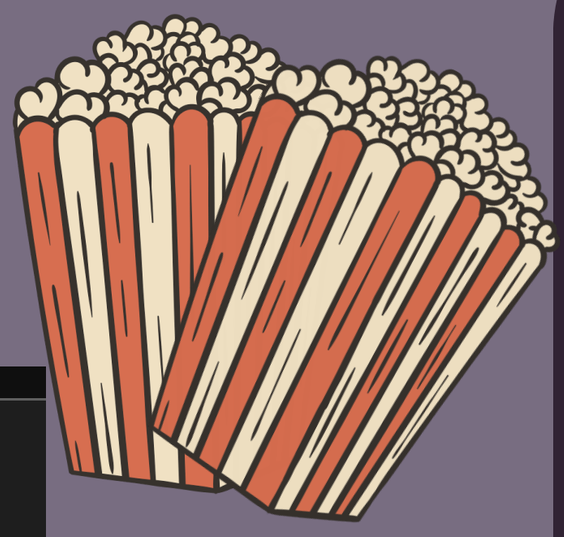
```python
df['review'] = df['review'].apply(remove_spaces)
```

Extra and unnecessary spaces were removed from the movie review dataset as part of the text cleaning and preprocessing stage in Natural Language Processing. A custom function named remove_spaces() was created to normalize whitespace by splitting the review text into individual words and then joining them back together using a single space separator. This process effectively removed multiple spaces, leading spaces, trailing spaces, and irregular spacing patterns present in the textual data. The function was applied to the review column using the apply() method to clean every movie review individually. Removing unnecessary spaces improved the consistency, readability, and formatting of the review text, ensuring that the dataset remained clean and well-structured for further NLP operations such as tokenization, stopword removal, stemming, TF-IDF vectorization, and machine learning-based sentiment classification.





TOKENIZATION



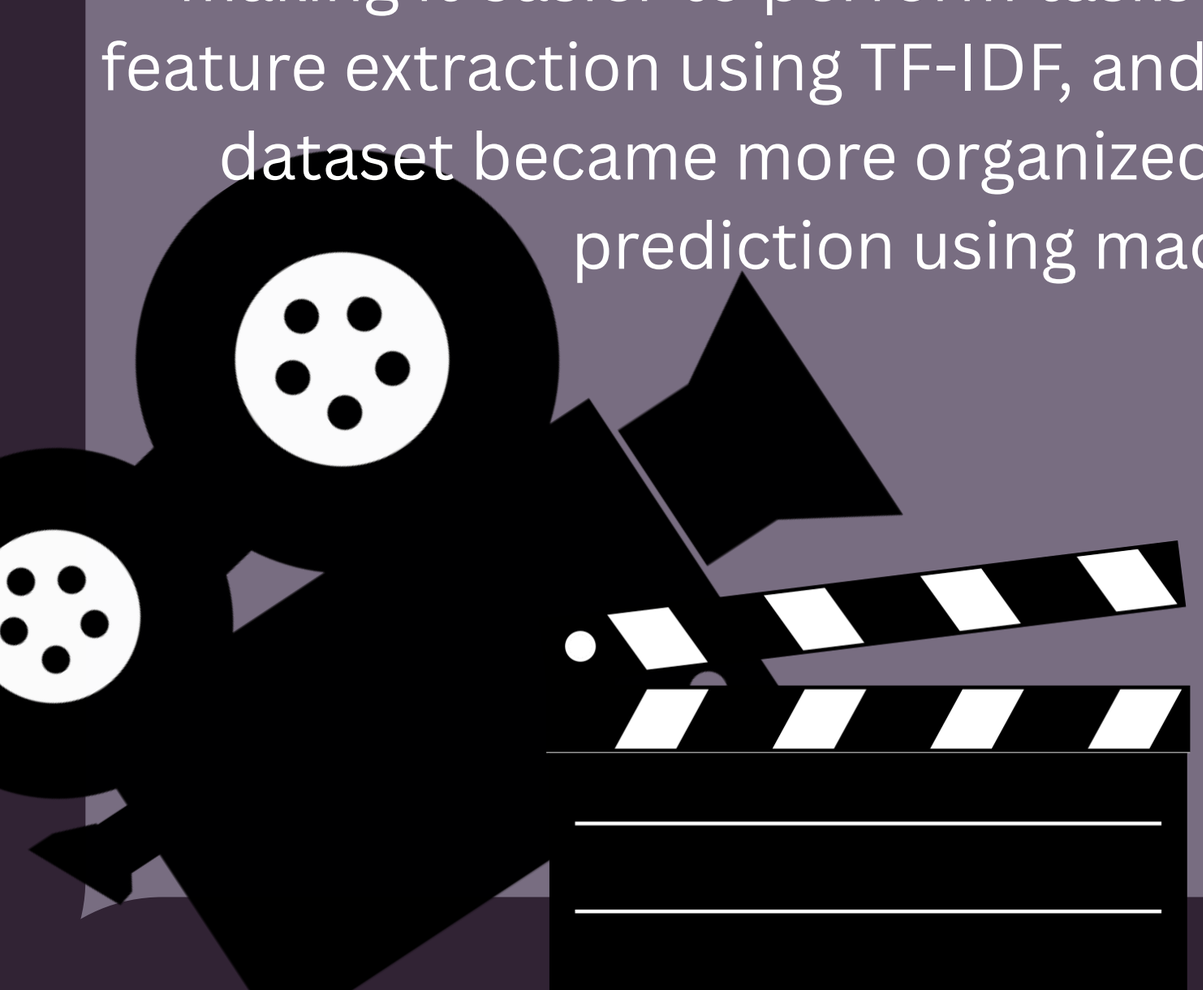
```
import nltk

nltk.download('punkt')
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\Apoorva\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\Apoorva\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

```
True
```

```
from nltk.tokenize import word_tokenize
import nltk

nltk.download('punkt')

df['review'] = df['review'].apply(word_tokenize)
```

Tokenization was performed during the Natural Language Processing preprocessing stage using the NLTK library to break movie review text into smaller individual units called tokens or words. The punkt tokenizer package was downloaded using nltk.download('punkt') to enable word-based tokenization. The word_tokenize() function from nltk.tokenize was then applied to the review column using the apply() method, converting each movie review into a list of individual words and meaningful text components. This preprocessing step helped structure the textual data in a format suitable for machine learning and further NLP operations. Tokenization improved text analysis by separating sentences into manageable units, making it easier to perform tasks such as stopword removal, stemming, feature extraction using TF-IDF, and sentiment classification. As a result, the dataset became more organized and suitable for accurate sentiment prediction using machine learning algorithms.

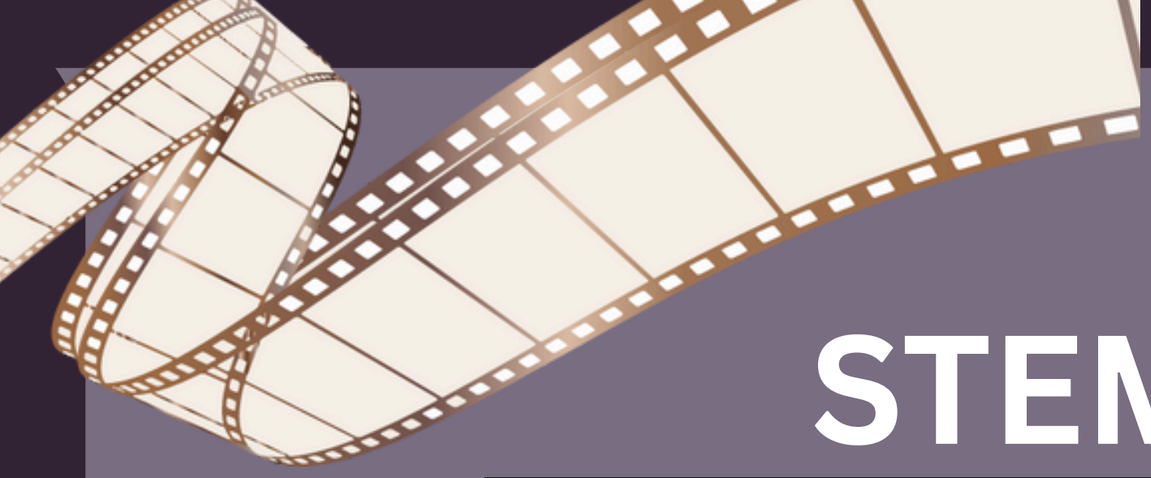


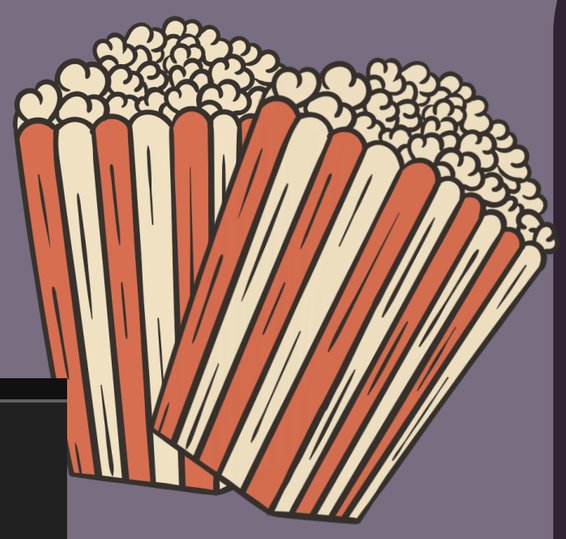
df

	review	sentiment
0	[one, reviewers, mentioned, watching, 1, oz, e...	positive
1	[wonderful, little, production, filming, techn...	positive
2	[thought, wonderful, way, spend, Tears, eyes, ...	positive
3	[basically, theres, family, little, boy, jake,...	negative
4	[petter, matteis, love, Tears, eyes, money, vi...	positive
...	...	...
49995	[thought, movie, right, good, job, wasnt, crea...	positive
49996	[bad, plot, bad, dialogue, bad, acting, idioti...	negative
49997	[catholic, taught, parochial, elementary, scho...	negative
49998	[im, going, disagree, previous, comment, side,...	negative
49999	[no, one, expects, star, trek, movies, high, a...	negative

49582 rows × 2 columns

The above output shows the movie review dataset after the tokenization process was applied using the word_tokenize() function from the NLTK library. After tokenization, each movie review was converted from a continuous text paragraph into a list of individual words or tokens. The review column now contains tokenized words enclosed within square brackets, while the sentiment column continues to store the corresponding sentiment labels as positive or negative. This transformation made the textual data more structured and machine-readable, allowing the model to process each word separately during feature extraction and sentiment analysis. The dataset contains 49,582 rows and 2 columns after preprocessing and duplicate removal. Tokenization played an important role in preparing the review text for further NLP operations such as stemming, TF-IDF vectorization, and machine learning model training for accurate movie review sentiment classification.

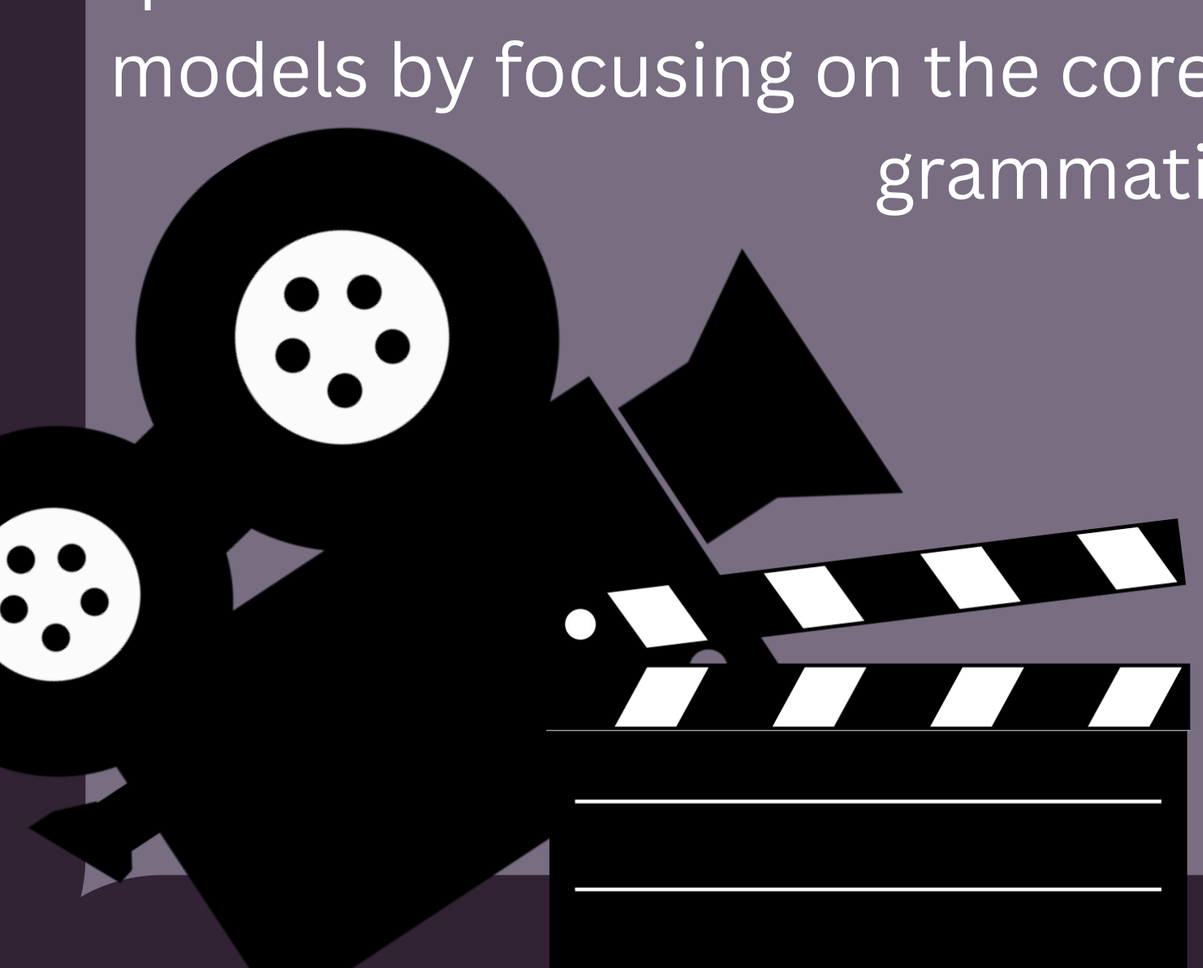




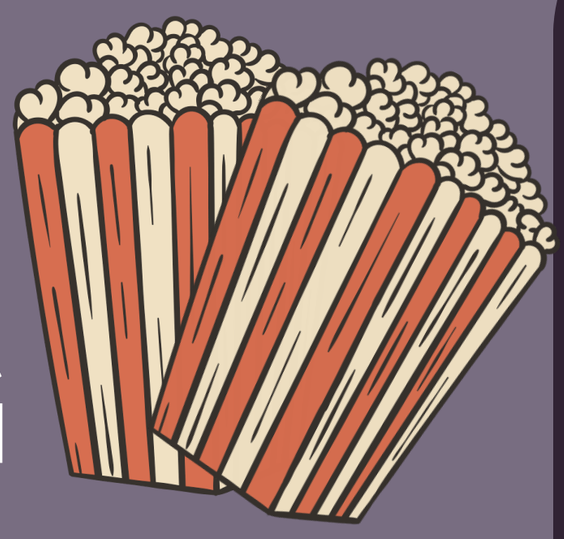
STEMMING

```python
from nltk.stem.porter import PorterStemmer

ps = PorterStemmer()

def stemming(words):
    return [ps.stem(word) for word in words]

df['review'] = df['review'].apply(stemming)
```

Stemming was performed during the Natural Language Processing preprocessing stage using the PorterStemmer algorithm from the NLTK library to reduce words to their root or base form. A PorterStemmer object was created and applied through a custom function named stemming(), which processed each tokenized word in the movie reviews individually. Words such as “watching”, “watched”, and “watches” were reduced to their common root form “watch”. This preprocessing step helped reduce vocabulary size, remove unnecessary word variations, and improve consistency in the textual dataset. After stemming, the processed tokens were joined back together into complete sentences using the " ".join() function so that the cleaned reviews could be used for feature extraction and machine learning model training. Stemming improved the efficiency of NLP analysis and enhanced the performance of TF-IDF vectorization and sentiment classification models by focusing on the core meaning of words rather than their grammatical variations.

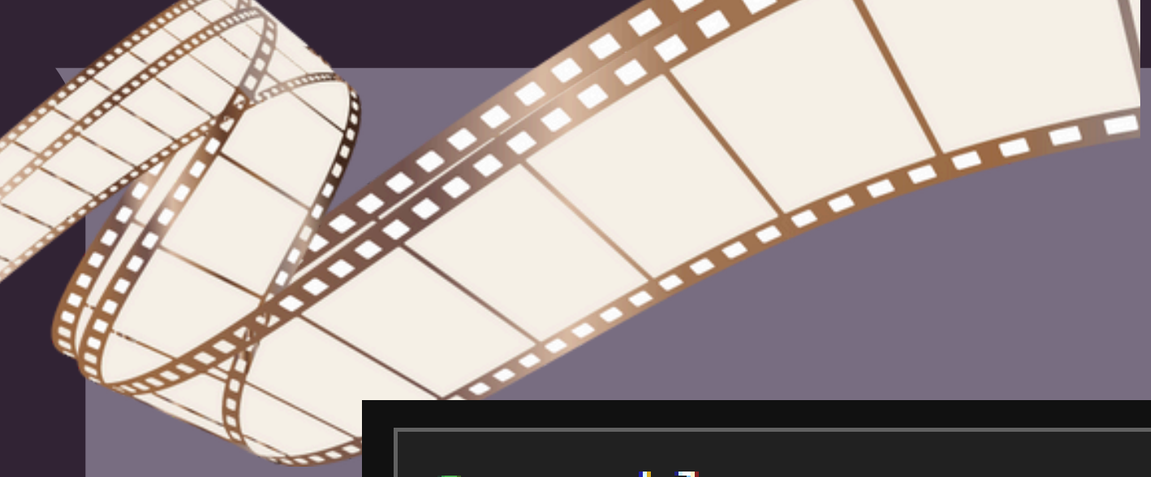


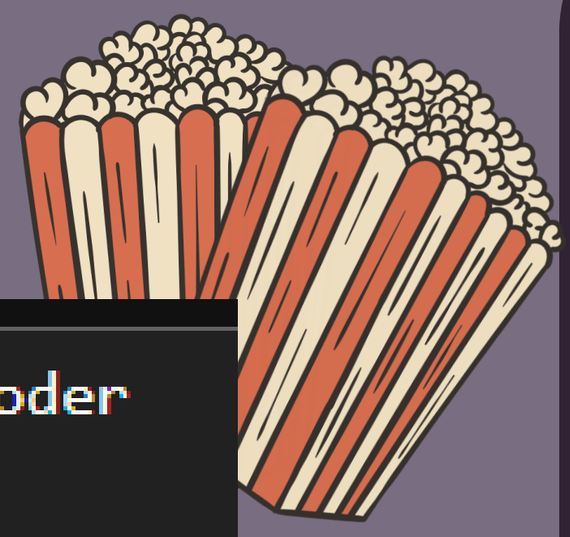
BOW AND LABEL ENCODING



```python
from sklearn.feature_extraction.text import CountVectorizer

cv = CountVectorizer(max_features=5000)

X = cv.fit_transform(df['review']).toarray()
```

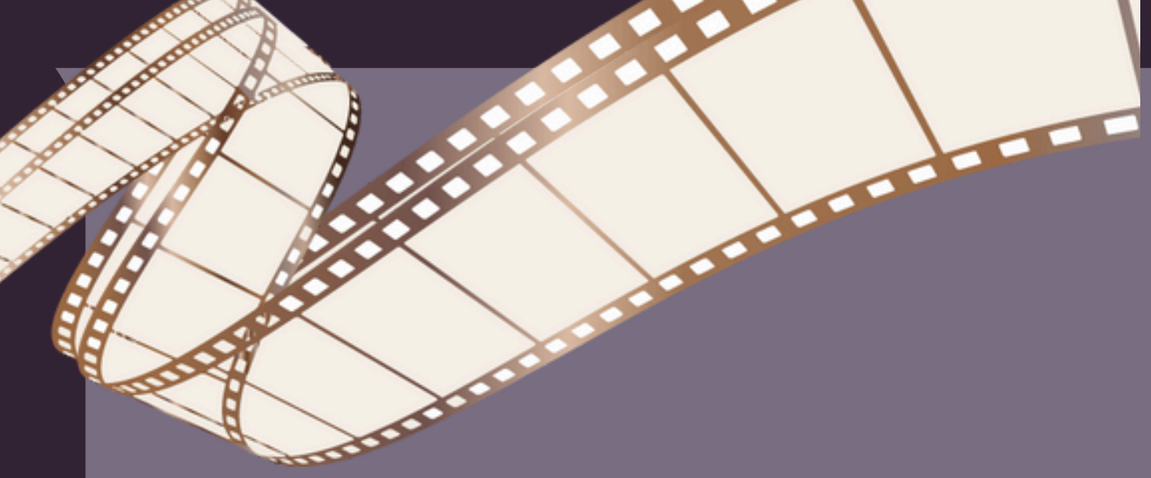
The Bag of Words (BOW) feature extraction technique was implemented using the CountVectorizer class from the sklearn.feature_extraction.text module to convert textual movie reviews into numerical data suitable for machine learning models. A CountVectorizer object was created with max_features=5000, which selected the 5000 most important and frequently occurring words from the dataset vocabulary. The fit_transform() function was then applied to the preprocessed review column to transform the textual data into a matrix of word frequency counts, where each row represented a movie review and each column represented a specific word feature. Finally, the sparse matrix output was converted into an array format using .toarray() for easier processing and model training. This feature extraction step enabled machine learning algorithms to understand textual information in numerical form by capturing the occurrence and importance of words present in movie reviews, thereby improving the effectiveness of sentiment classification.





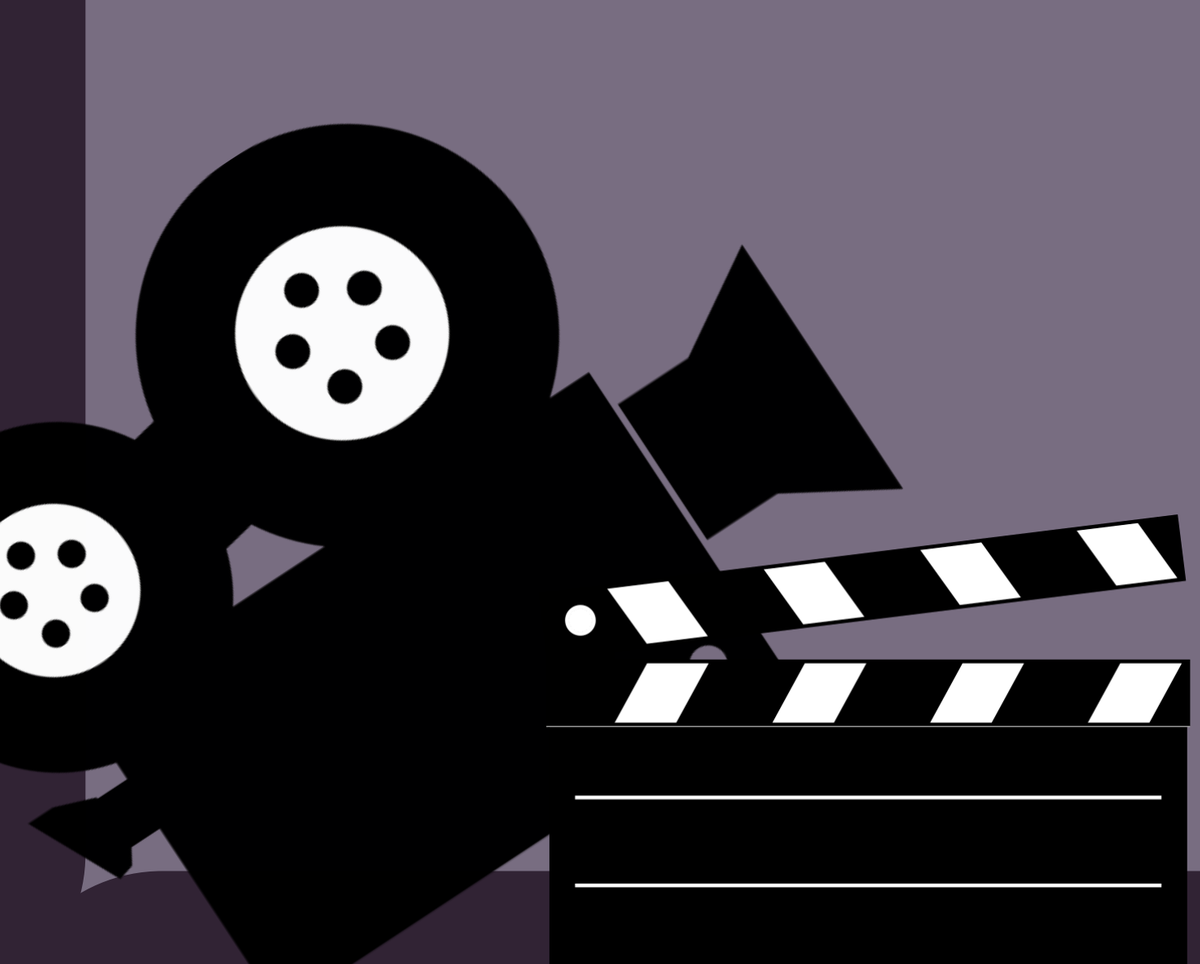
```python
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

y = le.fit_transform(df['sentiment'])
```

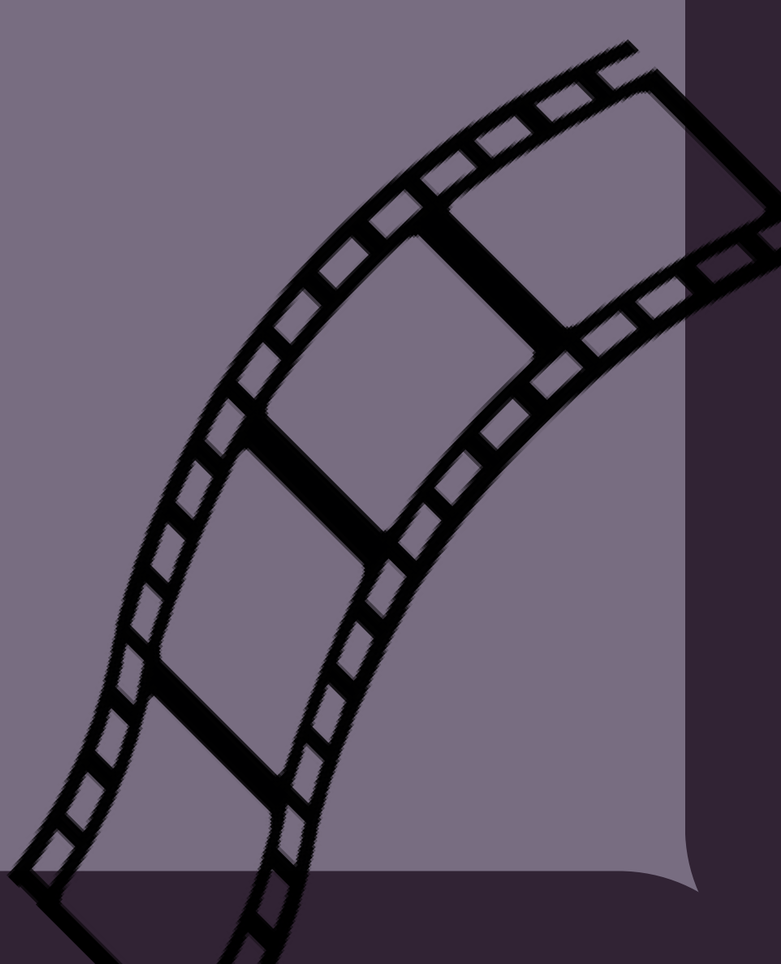
Label Encoding was performed using the LabelEncoder class from the sklearn.preprocessing module to convert categorical sentiment labels into numerical format suitable for machine learning algorithms. A LabelEncoder object was created and applied to the sentiment column using the fit_transform() function. This process transformed textual sentiment labels such as “positive” and “negative” into corresponding numerical values, allowing machine learning models to process the target variable effectively. The encoded output was stored in the variable y, which served as the dependent variable for sentiment classification. This preprocessing step was essential because machine learning algorithms require numerical input rather than textual labels for training and prediction. Label Encoding therefore helped prepare the dataset for accurate sentiment analysis and classification model development.

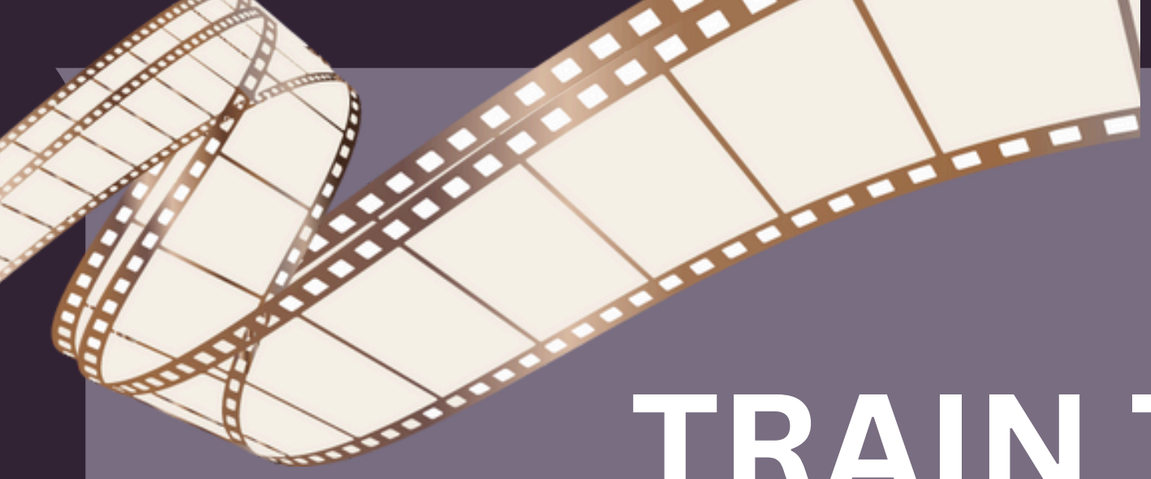


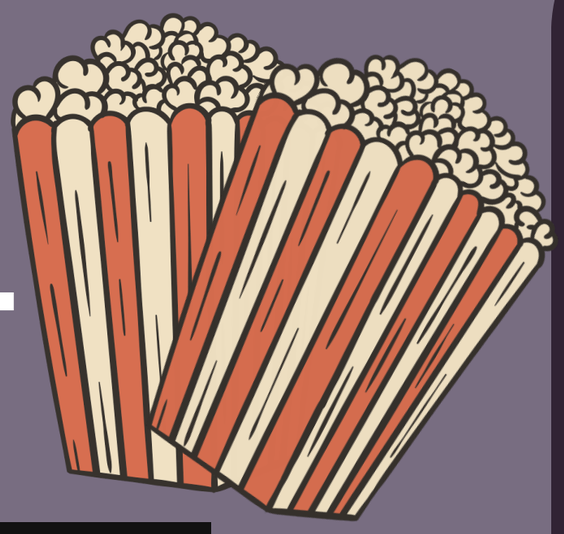
Intermission

Canva





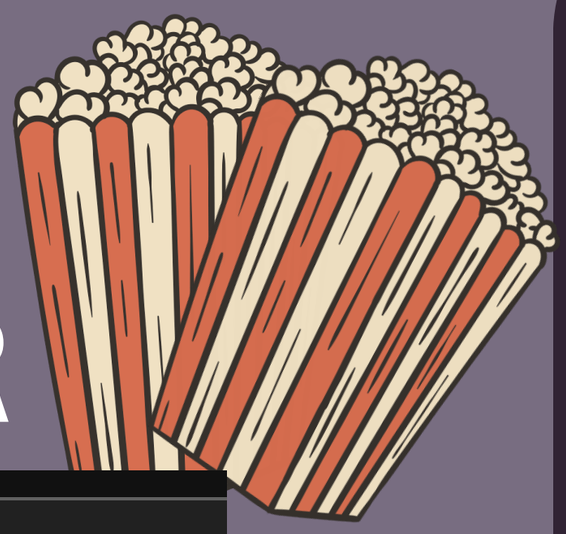




TRAIN TEST SPLIT

```python
from sklearn.model_selection import train_test_split
X = df['review']

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

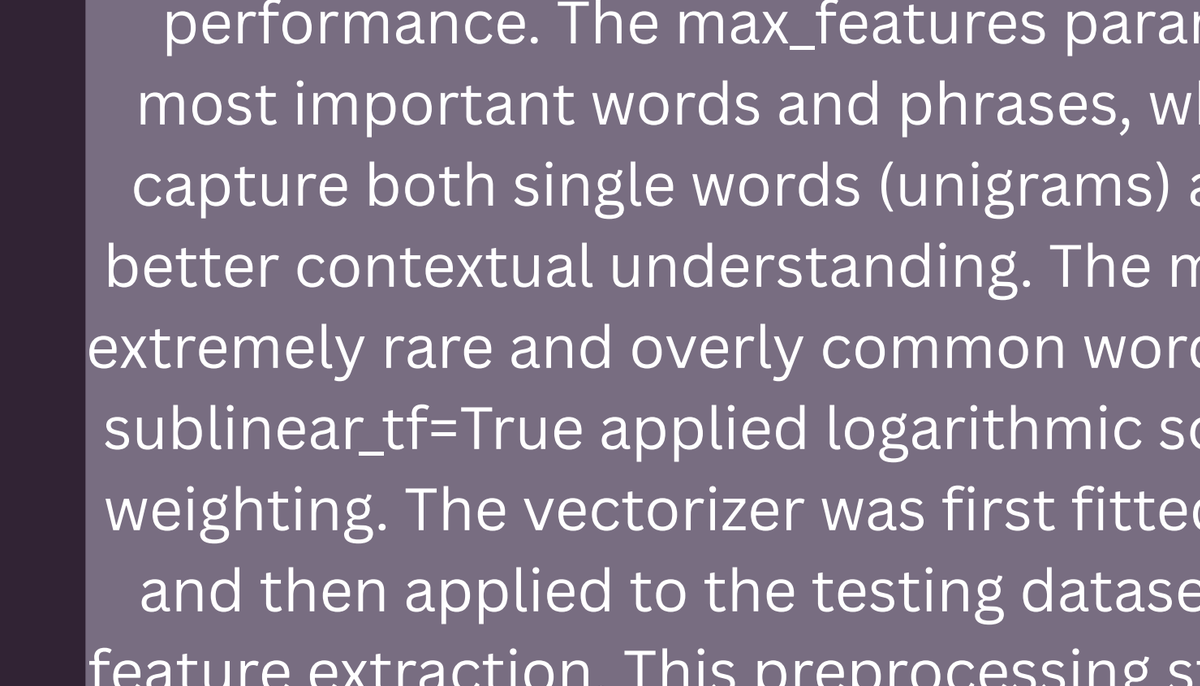
The dataset was divided into training and testing sets using the train_test_split() function from the sklearn.model_selection module to prepare the data for machine learning model development and evaluation. The review column was selected as the independent feature variable X, while the encoded sentiment labels stored in y were used as the target variable. The dataset was split with a test size of 20%, meaning that 80% of the data was used for training the machine learning models and 20% was reserved for testing and performance evaluation. The random_state=42 parameter was used to ensure reproducibility and consistency in the data splitting process. This step was important to evaluate how well the trained models could predict sentiments on unseen movie reviews and to measure their overall accuracy, reliability, and generalization performance.

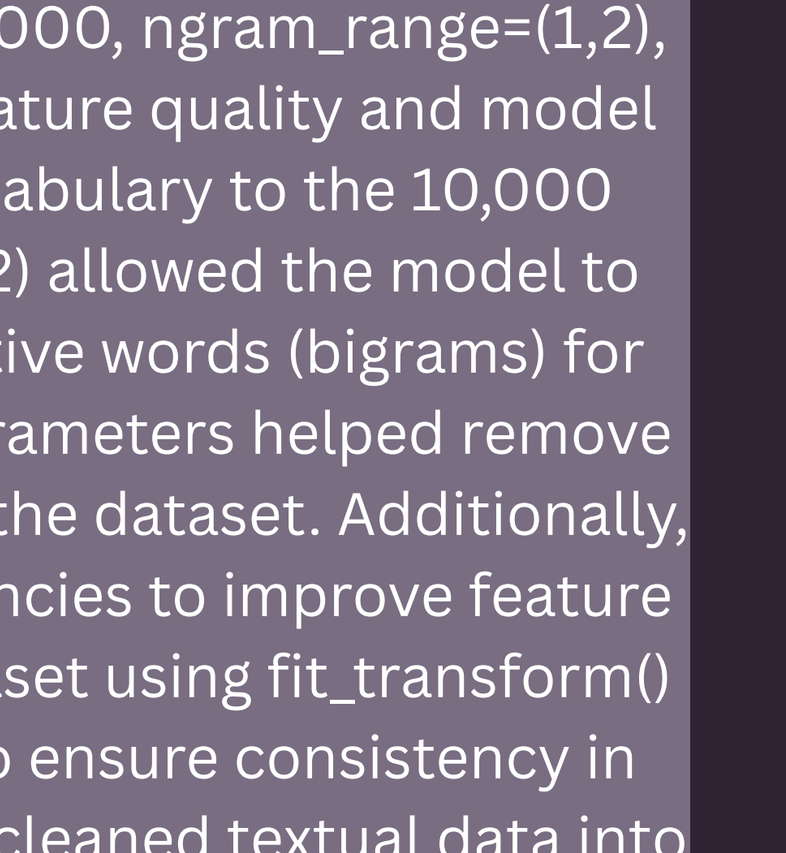


TF-ID VECTORIZER

```python
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(max_features=10000,
    ngram_range=(1,2),
    min_df=2,
    max_df=0.95,
    sublinear_tf=True)

X_train_tfidf = tfidf.fit_transform(X_train)

X_test_tfidf = tfidf.transform(X_test)
```

TF-IDF (Term Frequency–Inverse Document Frequency) vectorization was implemented using the TfidfVectorizer class from the sklearn.feature_extraction.text module to convert textual movie reviews into meaningful numerical feature representations for machine learning models. A TfidfVectorizer object was created with advanced parameter settings such as max_features=10000, ngram_range=(1,2), min_df=2, max_df=0.95, and sublinear_tf=True to improve feature quality and model performance. The max_features parameter limited the vocabulary to the 10,000 most important words and phrases, while ngram_range=(1,2) allowed the model to capture both single words (unigrams) and pairs of consecutive words (bigrams) for better contextual understanding. The min_df and max_df parameters helped remove extremely rare and overly common words, reducing noise in the dataset. Additionally, sublinear_tf=True applied logarithmic scaling to term frequencies to improve feature weighting. The vectorizer was first fitted on the training dataset using fit_transform() and then applied to the testing dataset using transform() to ensure consistency in feature extraction. This preprocessing step transformed the cleaned textual data into weighted numerical vectors, enabling machine learning algorithms to effectively understand word importance and perform accurate sentiment classification on movie reviews.





MACHINE LEARNING ALGORITHMS USED

Supervised Learning Algorithms

- Logistic Regression
- Multinomial NB
- Decision Tree Classifier
- Random Forest Classifier
- Linear Support Vector Classifier (Linear SVC)
- XGBoost (XGB) Classifier

Linear Models

- Logistic Regression
- Linear SVC

Tree-Based Algorithms

- Decision Tree Classifier
- Random Forest Classifier
- XGBoost Classifier

Bayesian Algorithm

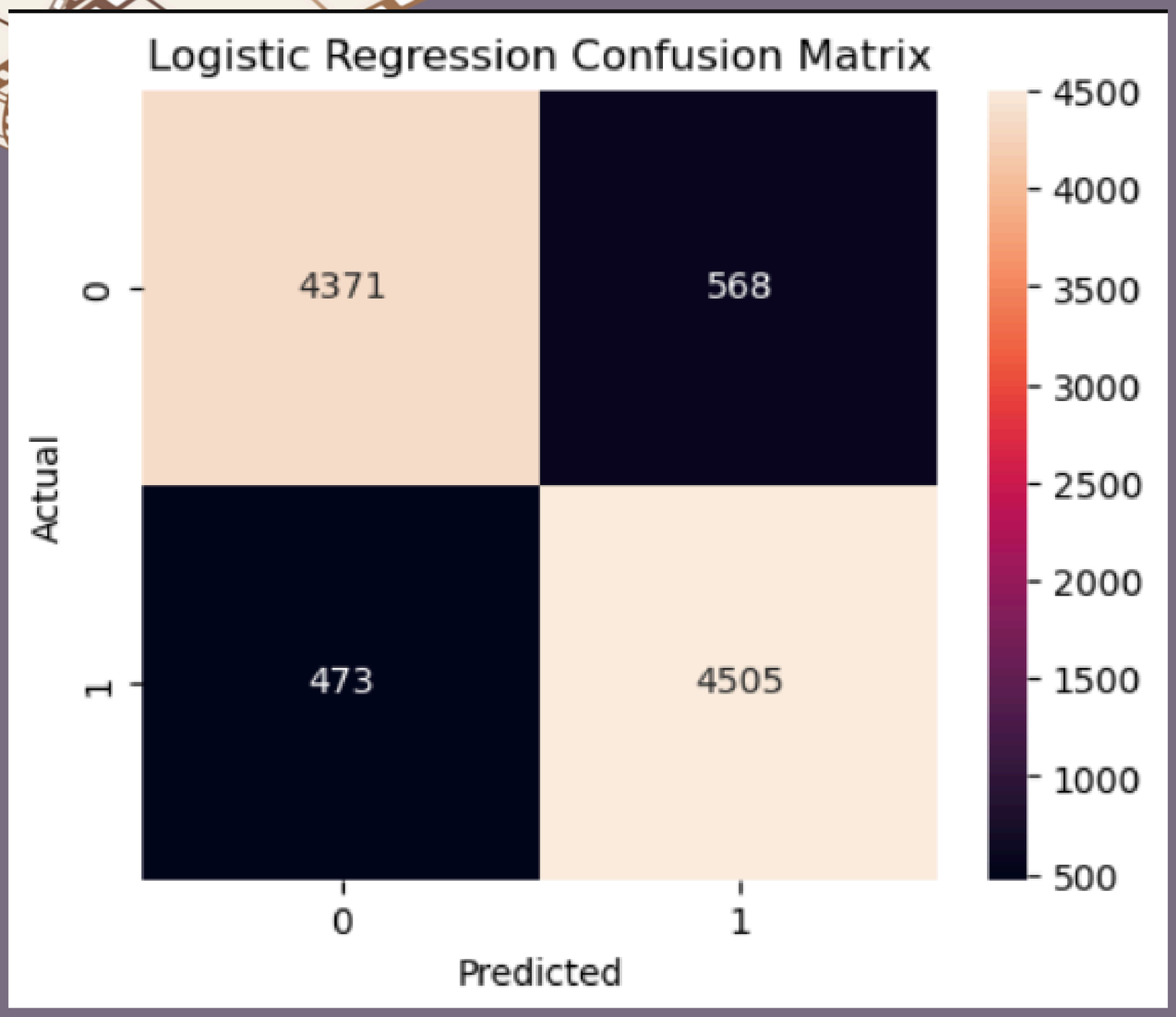
- Multinomial Naive Bayes

LOGISTIC REGRESSION

```
[43]: from sklearn.linear_model import LogisticRegression
      lr = LogisticRegression(C=4,
          max_iter=2000,
          solver='liblinear')

[44]: lr.fit(X_train_tfidf, y_train)

[44]: LogisticRegression
      LogisticRegression(C=4, max_iter=2000, solver='liblinear')

[45]: y_pred_lr = lr.predict(X_test_tfidf)

[46]: lr_acc = accuracy_score(y_test, y_pred_lr)

[47]: print("Logistic Regression Accuracy:", lr_acc)

      Logistic Regression Accuracy: 0.8950287385297974

[48]: print("\nClassification Report:\n")
      print(classification_report(y_test, y_pred_lr))

      Classification Report:

                    precision    recall  f1-score   support

                 0       0.90      0.88      0.89      4939
                 1       0.89      0.90      0.90      4978

          accuracy                           0.90      9917
         macro avg       0.90      0.89      0.90      9917
      weighted avg       0.90      0.90      0.90      9917

[83]: cm_lr = confusion_matrix(y_test, y_pred_lr)
      print("\nConfusion Matrix:\n")
      print(cm_lr)

      Confusion Matrix:

      [[4371  568]
       [ 473 4505]]
```

The image details the implementation and performance evaluation of a Logistic Regression model used for binary classification. The model was configured with a regularization parameter of C=4 and the 'liblinear' solver to ensure efficient convergence on the TF-IDF vectorized training data. Upon testing, the model demonstrated robust predictive power, achieving an overall accuracy of approximately 89.5%. This high performance is further validated by the classification report, which shows balanced precision, recall, and F1-scores of 0.90 for both classes.

Furthermore, the confusion matrix displays a high volume of correct predictions, with 4,371 true negatives and 4,505 true positives, indicating that the model is well-calibrated and effective at distinguishing between the two target categories.



The confusion matrix for the Logistic Regression model illustrates the classifier's strong performance in categorizing movie reviews. It shows that the model correctly identified 4,371 negative reviews (True Negatives) and 4,505 positive reviews (True Positives), reflecting a high level of accuracy across both sentiment classes. The visualization reveals a relatively low rate of error, with 568 instances where negative reviews were misclassified as positive and 473 instances where positive reviews were misclassified as negative. This balanced distribution of correct predictions suggests that the model is well-suited for the sentiment analysis task, maintaining consistent reliability in distinguishing between conflicting emotional tones in the dataset.

MULTINOMIAL NB

```python
from sklearn.naive_bayes import MultinomialNB
nb = MultinomialNB(alpha=0.3)
```

```python
nb.fit(X_train_tfidf, y_train)
```

```
MultinomialNB
MultinomialNB(alpha=0.3)
```

```python
y_pred_nb = nb.predict(X_test_tfidf)
```

```python
nb_acc = accuracy_score(y_test, y_pred_nb)

print("Naive Bayes Accuracy:", nb_acc)
```

```
Naive Bayes Accuracy: 0.8650801653725925
```

```python
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred_nb))
```

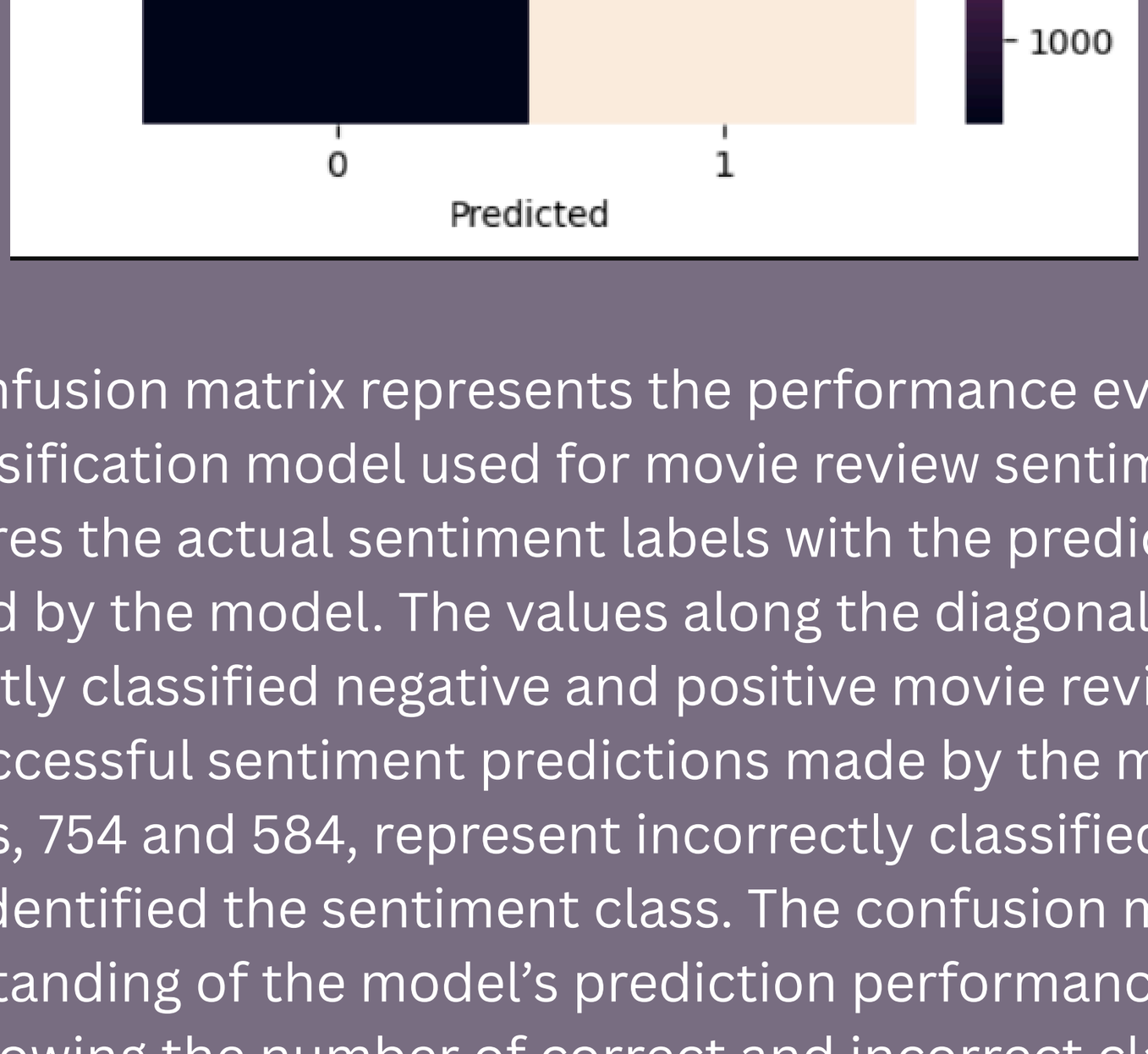
```
Classification Report:

              precision    recall  f1-score   support

           0       0.88      0.85      0.86      4939
           1       0.85      0.88      0.87      4978

    accuracy                           0.87      9917
   macro avg       0.87      0.87      0.87      9917
weighted avg       0.87      0.87      0.87      9917
```

```python
cm_nb = confusion_matrix(y_test, y_pred_nb)

print("\nConfusion Matrix:\n")
print(cm_nb)
```

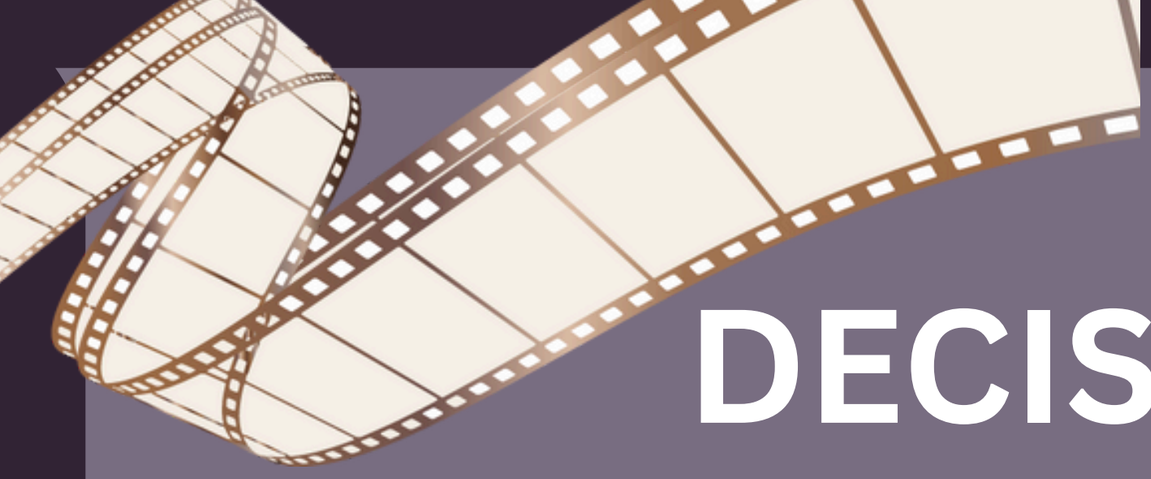
```
Confusion Matrix:

[[4185  754]
 [ 584 4394]]
```

The Multinomial Naive Bayes classifier was implemented as an alternative modeling approach, utilizing an alpha value of 0.3 for Laplace smoothing to optimize performance on the movie review dataset. This model achieved a respectable accuracy of approximately 86.5%, demonstrating its effectiveness in processing TF-IDF vectorized text. The classification report reveals a strong balance between precision and recall for both sentiment classes, with F1-scores of 0.86 and 0.87 for negative and positive reviews, respectively.

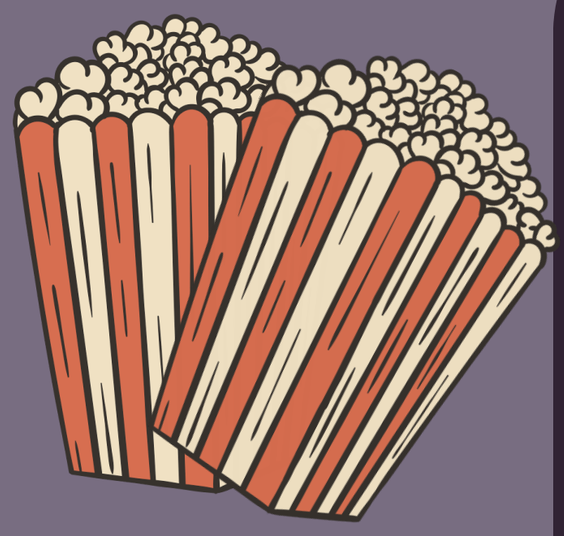
According to the confusion matrix, the model correctly predicted 4,185 negative and 4,394 positive reviews, showing slightly more difficulty in distinguishing false positives compared to the Logistic Regression baseline but remaining a computationally efficient and reliable predictor for sentiment analysis.



The above confusion matrix represents the performance evaluation of the Naive Bayes classification model used for movie review sentiment analysis. The matrix compares the actual sentiment labels with the predicted sentiment labels generated by the model. The values along the diagonal, 4185 and 4394, represent correctly classified negative and positive movie reviews respectively, indicating successful sentiment predictions made by the model. The off-diagonal values, 754 and 584, represent incorrectly classified reviews where the model misidentified the sentiment class. The confusion matrix provides a detailed understanding of the model's prediction performance beyond overall accuracy by showing the number of correct and incorrect classifications for each sentiment category. The heatmap visualization helps analyze the effectiveness of the Naive Bayes algorithm in distinguishing between positive and negative movie reviews and demonstrates that the model achieved good classification performance on the dataset.



DECISIONTREE CLASSIFIER



```
from sklearn.tree import DecisionTreeClassifier
dt = DecisionTreeClassifier(criterion='gini',
    max_depth=100,
    min_samples_split=20,min_samples_leaf=10, random_state=42)
```

```
dt.fit(X_train_tfidf, y_train)
```

```
DecisionTreeClassifier(max_depth=100, min_samples_leaf=10, min_samples_split=20,
                       random_state=42)
```

```
y_pred_dt = dt.predict(X_test_tfidf)
```

```
dt_acc = accuracy_score(y_test, y_pred_dt)
print("Decision Tree Accuracy:", dt_acc)
```

```
Decision Tree Accuracy: 0.7266310376121811
```

```
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred_dt))
```

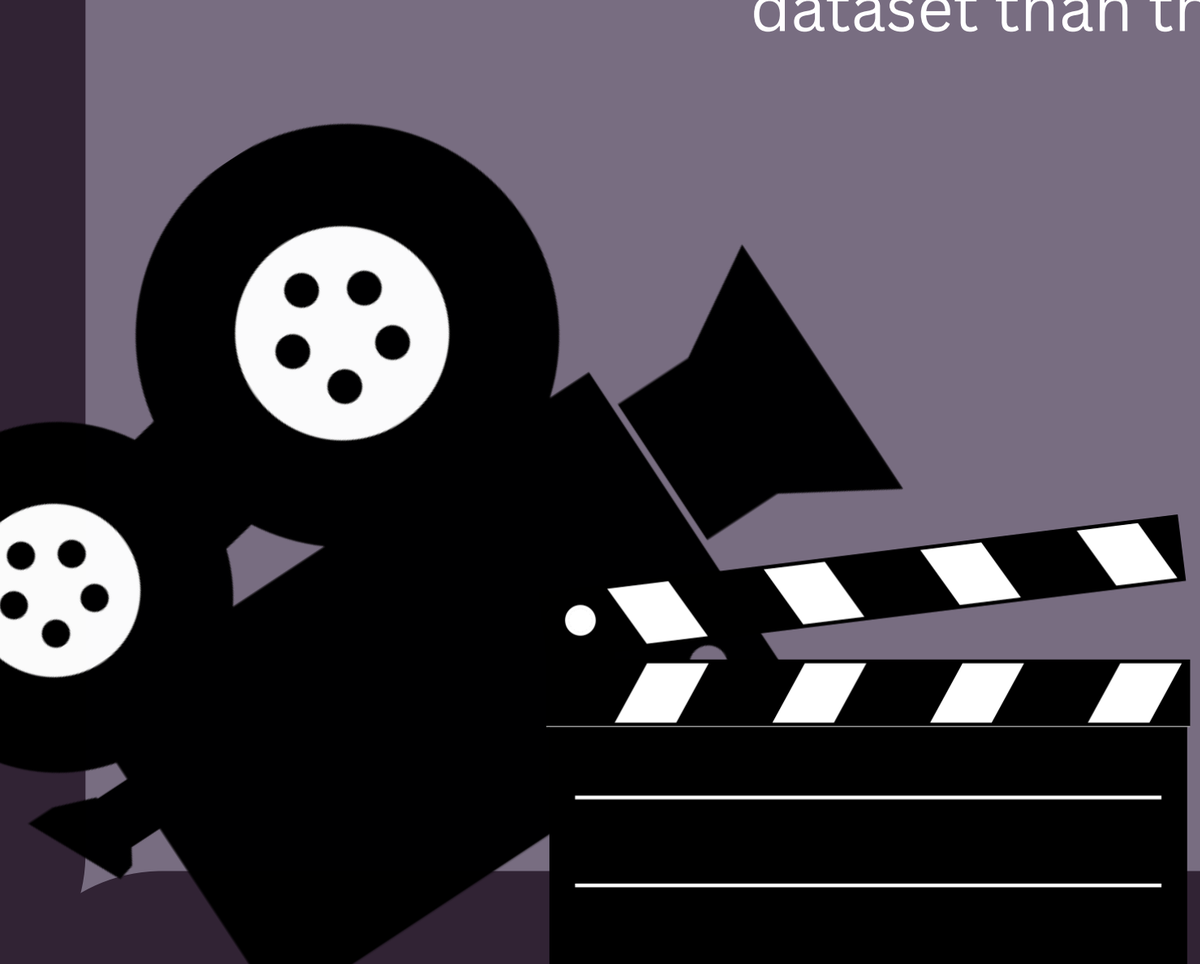
```
Classification Report:

              precision    recall  f1-score   support

           0       0.72      0.74      0.73      4939
           1       0.73      0.72      0.73      4978

    accuracy                           0.73      9917
   macro avg       0.73      0.73      0.73      9917
weighted avg       0.73      0.73      0.73      9917
```

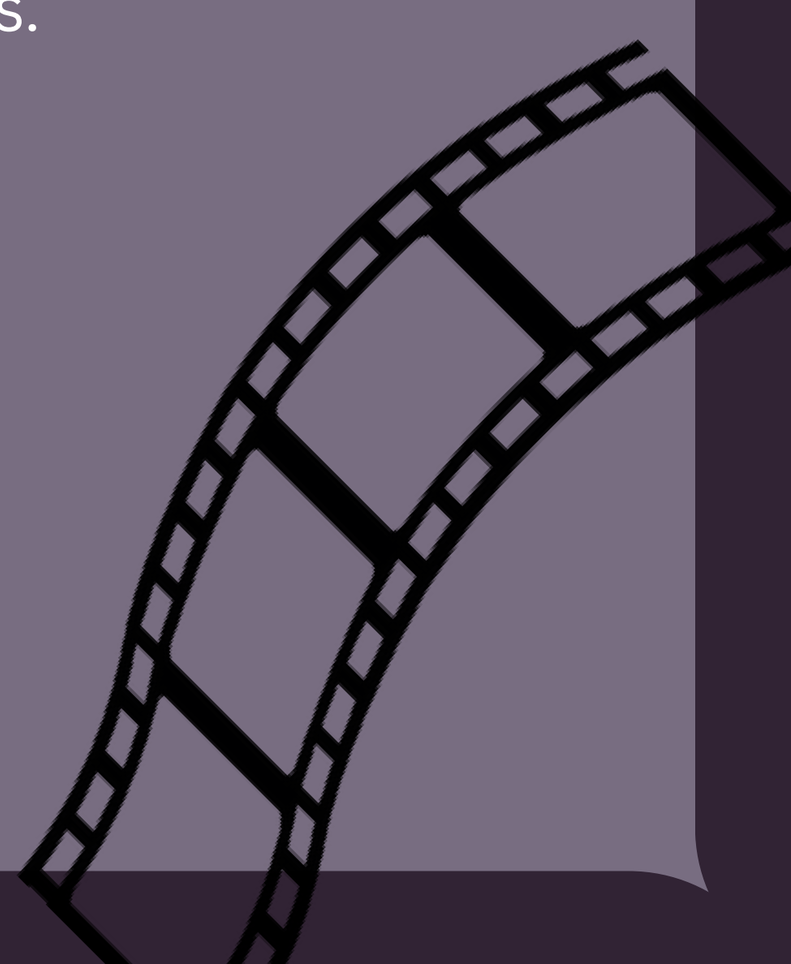
```
cm_dt = confusion_matrix(y_test, y_pred_dt)
print("\nConfusion Matrix:\n")
print(cm_dt)
```

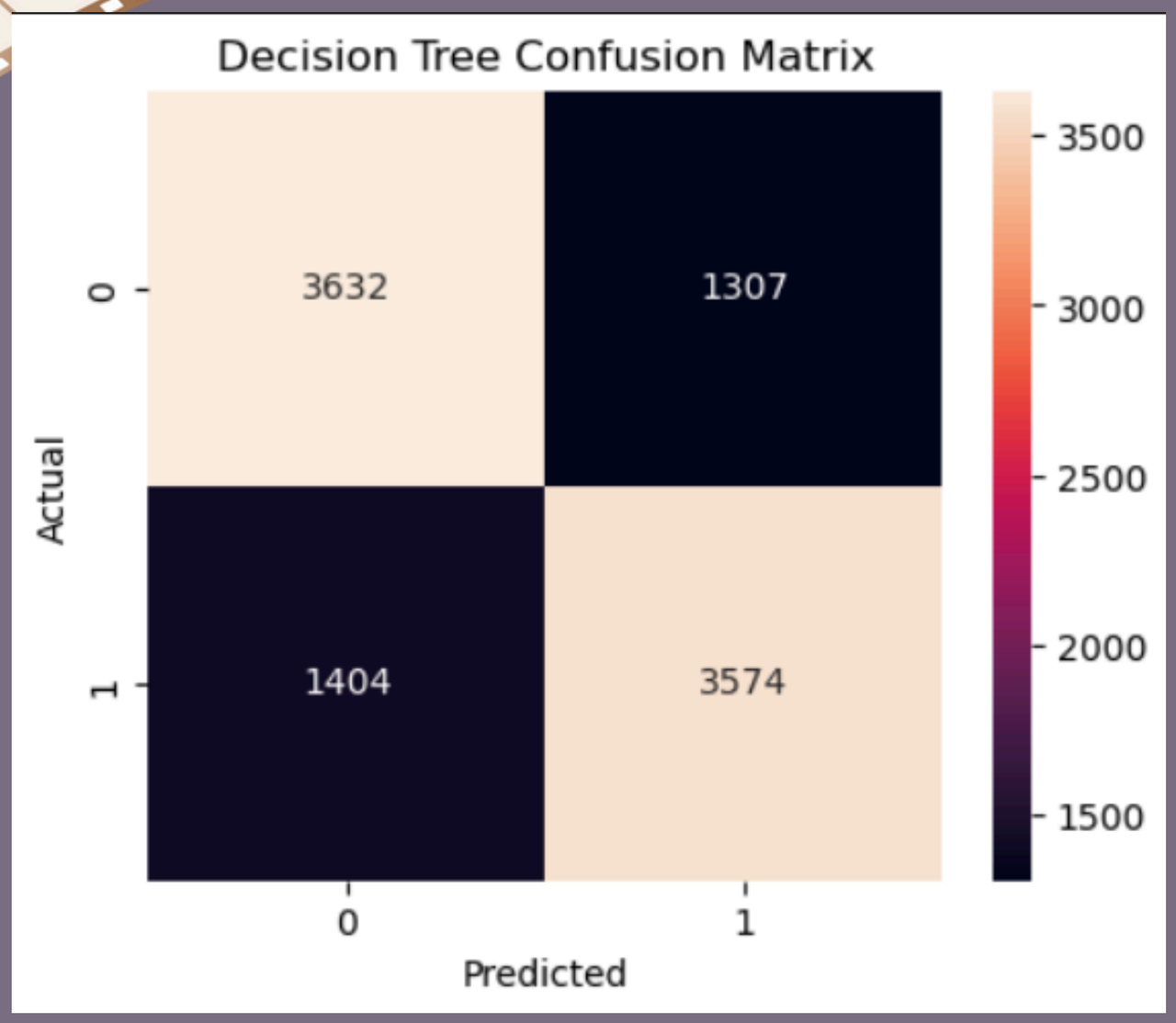
```
Confusion Matrix:

[[3632 1307]
 [1404 3574]]
```

A Decision Tree classifier was also implemented, utilizing specific hyperparameters such as a maximum depth of 100 and minimum sample requirements for splits and leaves to manage model complexity and prevent overfitting. This approach yielded an accuracy of approximately 72.7%, indicating a lower performance threshold compared to the Logistic Regression and Naive Bayes models. The classification report reflects a balanced but more moderate predictive capability, with F1-scores, precision, and recall consistently hovering around 0.73 for both sentiment categories.

The confusion matrix reveals 3,632 true negatives and 3,574 true positives, though the higher volume of misclassifications—1,307 false positives and 1,404 false negatives—suggests that the hierarchical structure of a decision tree may be less efficient at capturing the nuances of this specific TF-IDF vectorized dataset than the linear alternatives.







The confusion matrix for the Decision Tree model provides a visual representation of its performance in classifying movie review sentiment. The model successfully predicted 3,632 negative reviews and 3,574 positive reviews. However, the matrix highlights a higher error margin compared to other classifiers, with 1,307 instances where negative sentiment was misidentified as positive and 1,404 instances where positive reviews were misclassified as negative. These results indicate that while the Decision Tree approach maintains a degree of balance, it is significantly more prone to misclassification within this specific NLP pipeline.

RANDOMFOREST

```python
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=200,
    max_depth=50,
    min_samples_split=5,random_state=42,n_jobs=-1)
```

```python
rf.fit(X_train_tfidf, y_train)
```

```
RandomForestClassifier(max_depth=50, min_samples_split=5, n_estimators=200,
                       n_jobs=-1, random_state=42)
```

```python
y_pred_rf = rf.predict(X_test_tfidf)
```

```python
rf_acc = accuracy_score(y_test, y_pred_rf)
print("Random Forest Accuracy:", rf_acc)
```

```
Random Forest Accuracy: 0.845114449934456
```

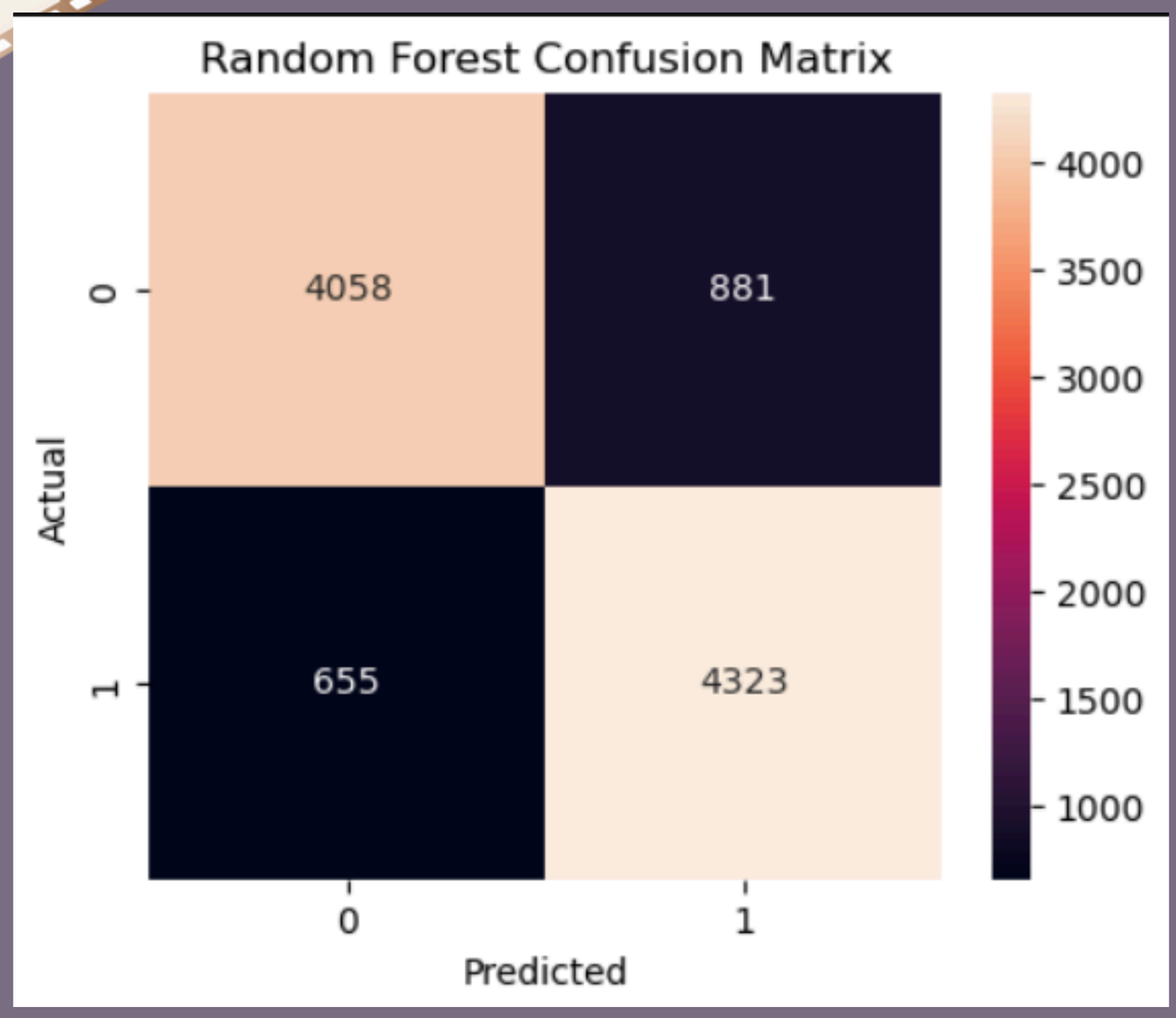
```python
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred_rf))
```

```
Classification Report:

              precision    recall  f1-score   support

           0       0.86      0.82      0.84      4939
           1       0.83      0.87      0.85      4978

    accuracy                           0.85      9917
   macro avg       0.85      0.85      0.85      9917
weighted avg       0.85      0.85      0.85      9917
```

```python
cm_rf = confusion_matrix(y_test, y_pred_rf)
print("\nConfusion Matrix:\n")
print(cm_rf)
```

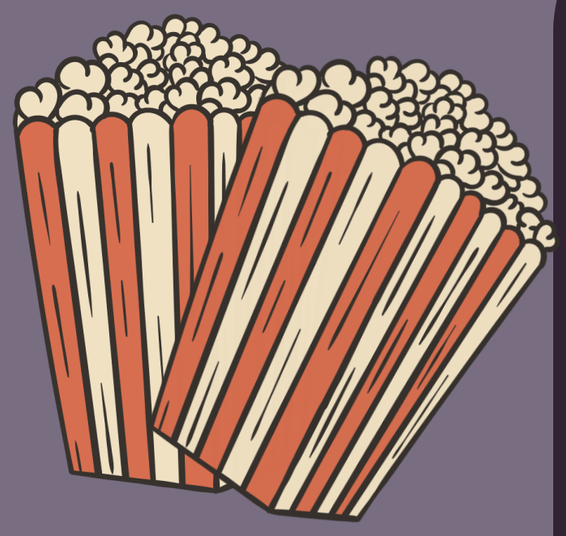
```
Confusion Matrix:

[[4058  881]
 [ 655 4323]]
```

The Random Forest Classifier was implemented using 200 estimators and a maximum depth of 50 to provide a robust ensemble approach for sentiment classification. This model achieved an overall accuracy of approximately 84.5% on the test dataset. According to the classification report, the model maintained balanced performance across both sentiment categories, yielding an F1-score of 0.84 for negative reviews and 0.85 for positive reviews. The confusion matrix further details this performance, showing 4,058 correctly identified negative reviews and 4,323 correctly identified positive reviews. While the model produced 881 false positives and 655 false negatives, its high precision and recall scores demonstrate its effectiveness as a reliable predictor for this NLP task.



The confusion matrix for the Random Forest model provides a clear visual summary of its sentiment classification performance. The model successfully categorized 4,058 reviews as negative (True Negatives) and 4,323 reviews as positive (True Positives). The visualization reveals that the classifier encountered 881 instances where negative sentiment was incorrectly predicted as positive and 655 instances where positive reviews were mislabeled as negative. Overall, the high density of correct predictions along the main diagonal confirms that the ensemble-based approach is robust and maintains a high degree of reliability in distinguishing between positive and negative movie reviews.

LINEAR SVC



```
from sklearn.svm import LinearSVC
svm = LinearSVC(C=1.5,max_iter=5000)
```

```
svm.fit(X_train_tfidf, y_train)
```

```
LinearSVC
LinearSVC(C=1.5, max_iter=5000)
```

```
y_pred_svm = svm.predict(X_test_tfidf)
```

```
svm_acc = accuracy_score(y_test, y_pred_svm)
print("SVM Accuracy:", svm_acc)
```

```
SVM Accuracy: 0.8837350005041847
```

```
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred_svm))
```

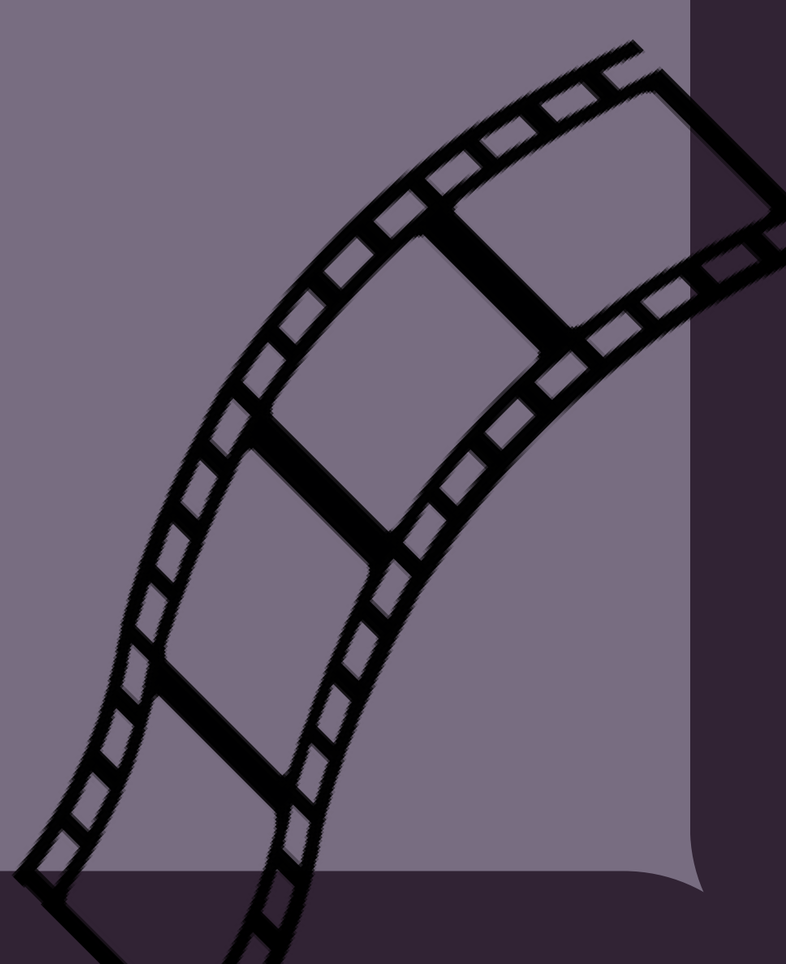
```
Classification Report:

              precision    recall  f1-score   support

           0       0.89      0.88      0.88      4939
           1       0.88      0.89      0.89      4978

    accuracy                           0.88      9917
   macro avg       0.88      0.88      0.88      9917
weighted avg       0.88      0.88      0.88      9917
```

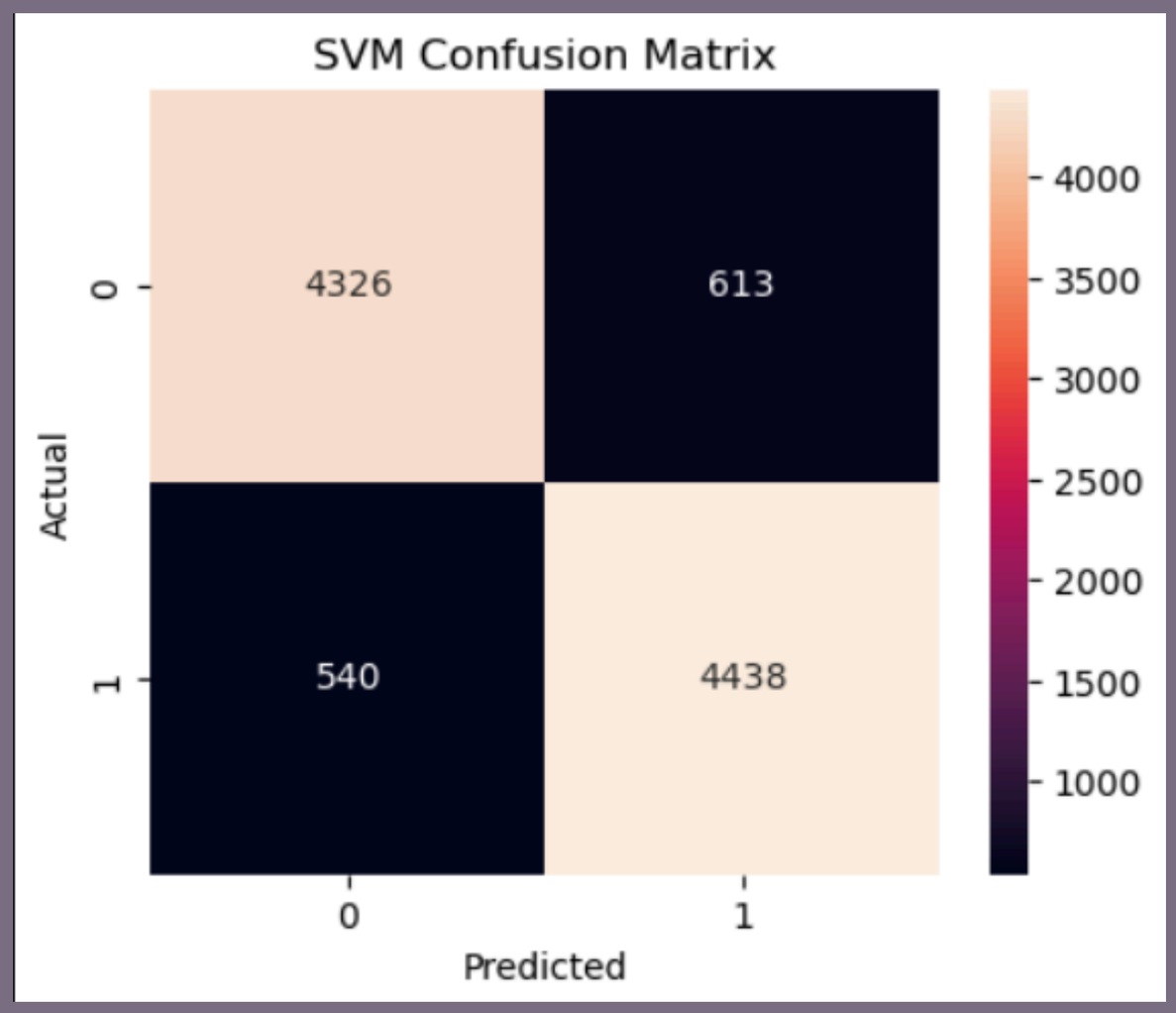
```
cm_svm = confusion_matrix(y_test, y_pred_svm)
print("\nConfusion Matrix:\n")
print(cm_svm)
```

```
Confusion Matrix:

[[4326  613]
 [ 540 4438]]
```

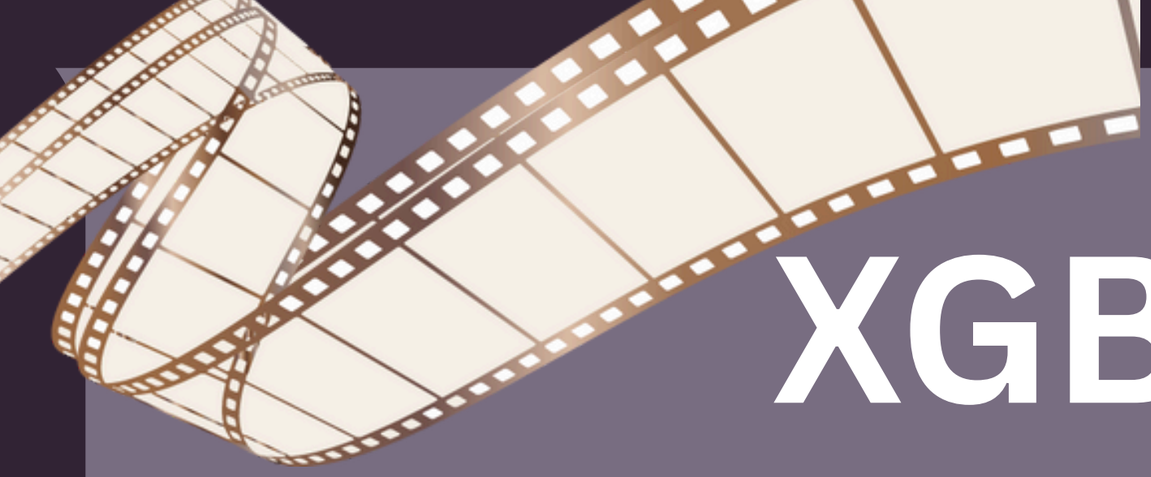
The Support Vector Machine (SVM) model, implemented using the LinearSVC algorithm with a regularization parameter of C=1.5, achieved a strong accuracy of approximately 88.4%. The classification report indicates high consistency across both sentiment classes, with precision, recall, and F1-scores all maintaining a range between 0.88 and 0.89. This balanced performance is further supported by the confusion matrix, which shows 4,326 correct negative predictions and 4,438 correct positive predictions.

With relatively low counts of false positives (613) and false negatives (540), the SVM proves to be one of the most effective models in this NLP pipeline for accurately separating sentiment in movie reviews.

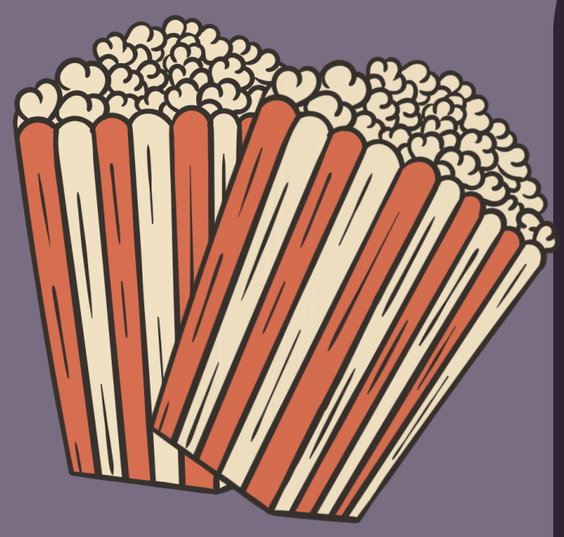




The SVM confusion matrix illustrates the model's strong capability in distinguishing between positive and negative sentiments in movie reviews. It reveals that the classifier correctly identified 4,326 negative reviews (True Negatives) and 4,438 positive reviews (True Positives). The visualization indicates a high level of precision, with only 613 false positives and 540 false negatives recorded across the test set. This balanced distribution of correct predictions along the diagonal highlights the effectiveness of the Support Vector Machine in handling the linguistic complexities of the dataset with minimal classification error.



XGBOOST CLASSIFIER



```python
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score
xgb = XGBClassifier(
    n_estimators=200,
    learning_rate=0.1,
    max_depth=6,
    random_state=42,
    n_jobs=-1
)
xgb.fit(X_train_tfidf, y_train)

y_pred_xgb = xgb.predict(X_test_tfidf)

xgb_acc = accuracy_score(y_test, y_pred_xgb)

print("XGBoost Accuracy:", xgb_acc)
```

```
XGBoost Accuracy: 0.8466270041343148
```

```python
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred_xgb))
```

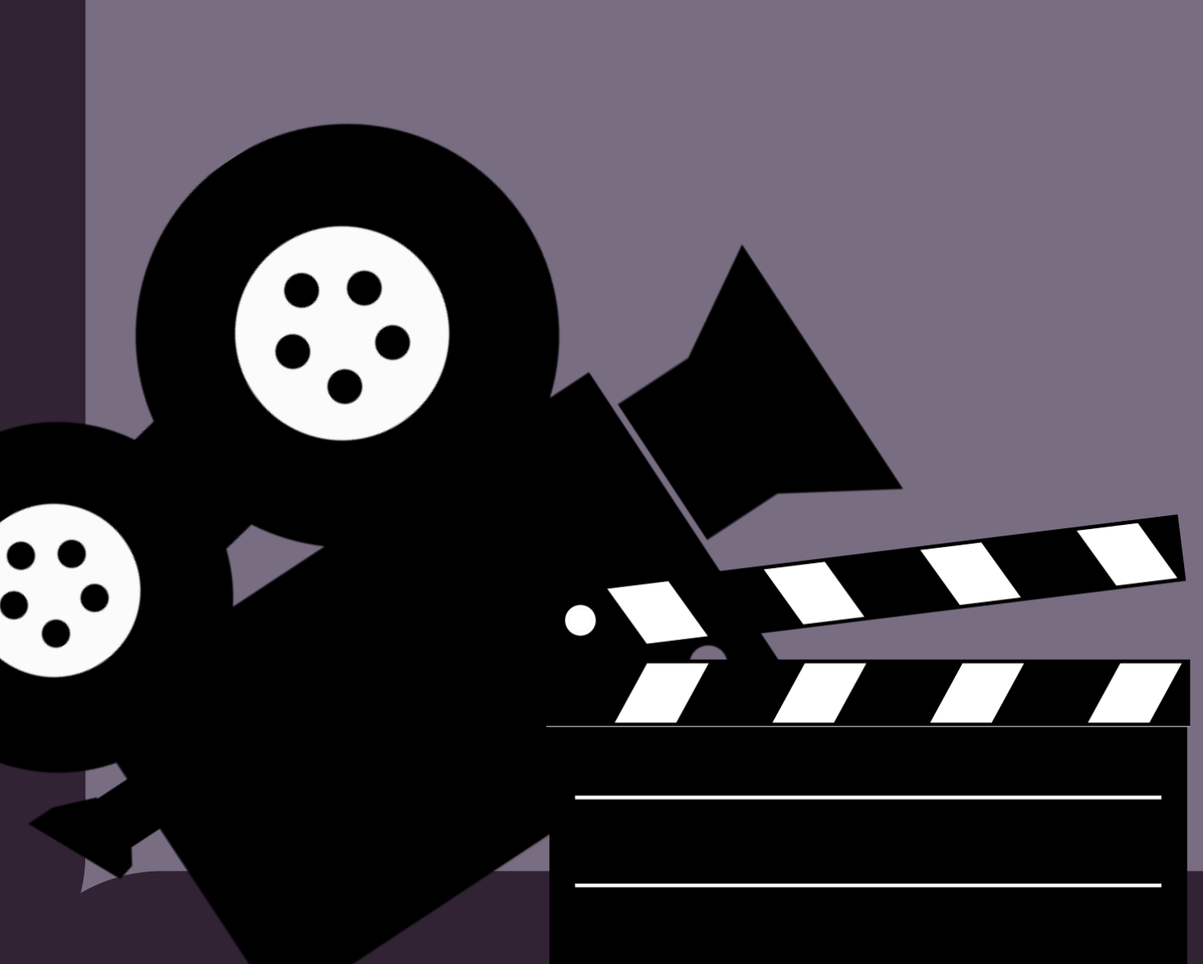
```
Classification Report:

              precision    recall  f1-score   support

           0       0.87      0.82      0.84      4939
           1       0.83      0.87      0.85      4978

    accuracy                           0.85      9917
   macro avg       0.85      0.85      0.85      9917
weighted avg       0.85      0.85      0.85      9917
```

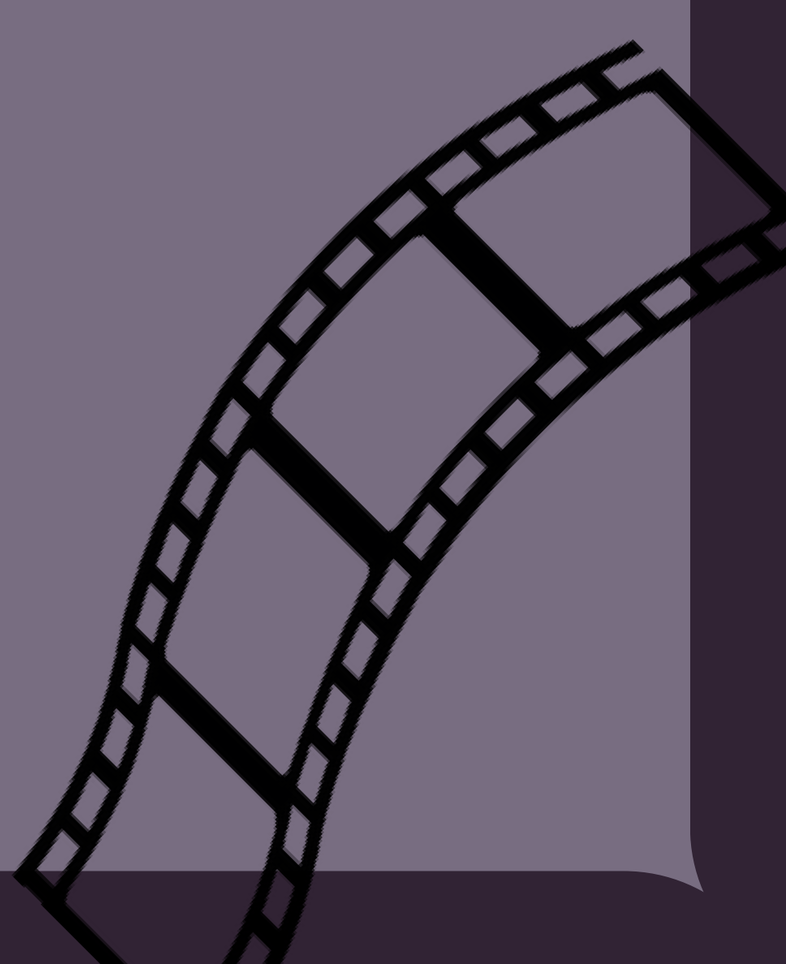
```python
cm_xgb = confusion_matrix(y_test, y_pred_xgb)
print("\nConfusion Matrix:\n")
print(cm_xgb)
```

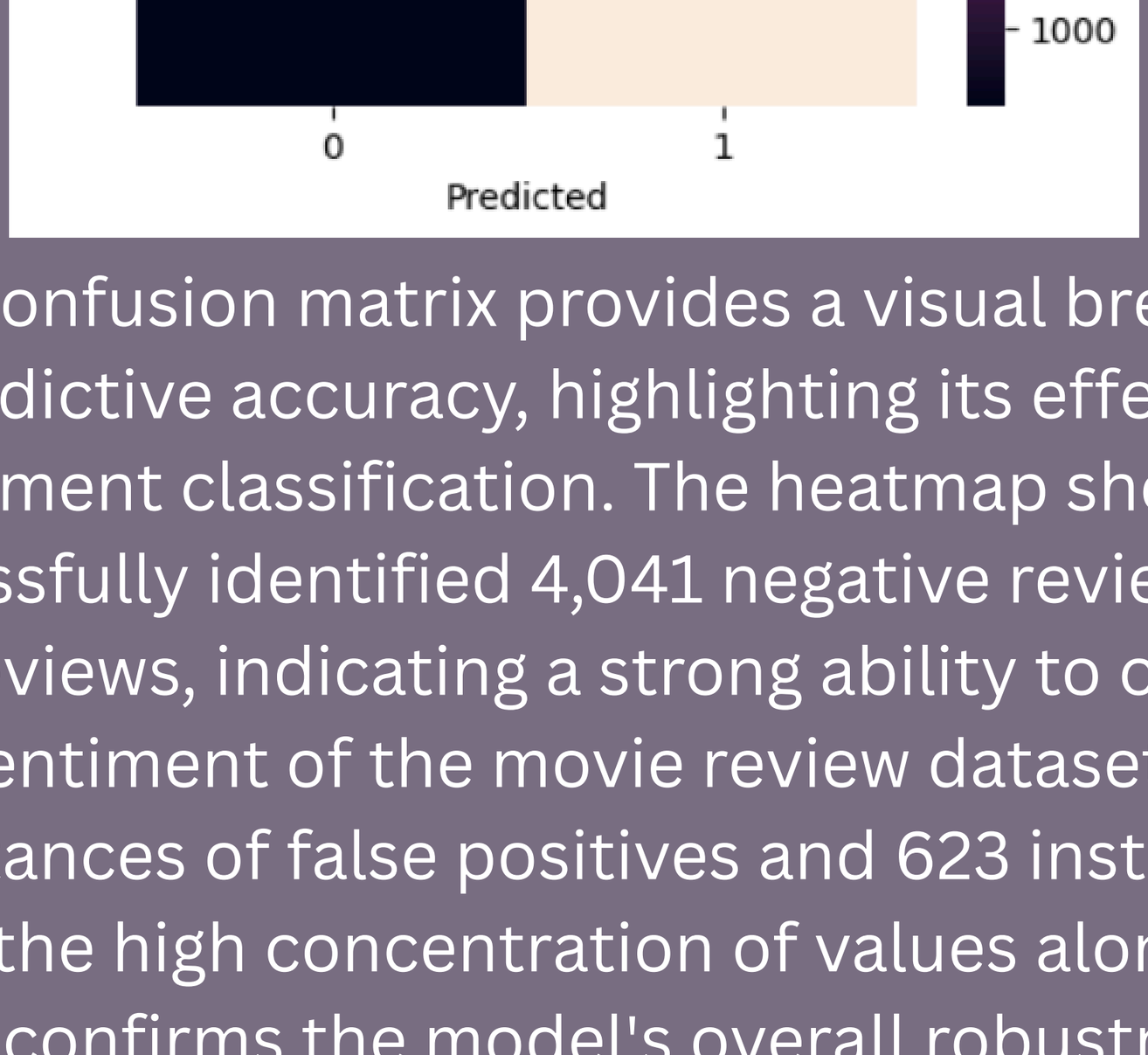
```
Confusion Matrix:

[[4041  898]
 [ 623 4355]]
```

The XGBoost classifier was trained using 200 estimators and a learning rate of 0.1 to provide a gradient-boosted ensemble perspective on the movie review data. The model achieved an accuracy of approximately 84.7%, performing similarly to the Random Forest approach. Detailed metrics from the classification report show an F1-score of 0.84 for negative reviews and 0.85 for positive reviews, indicating a high level of predictive consistency. The confusion matrix further breaks down these results, identifying 4,041 true negatives and 4,355 true positives, while managing 898 false positives and 623 false negatives.

This performance confirms that XGBoost is a highly effective and robust model for capturing complex patterns within TF-IDF vectorized text.

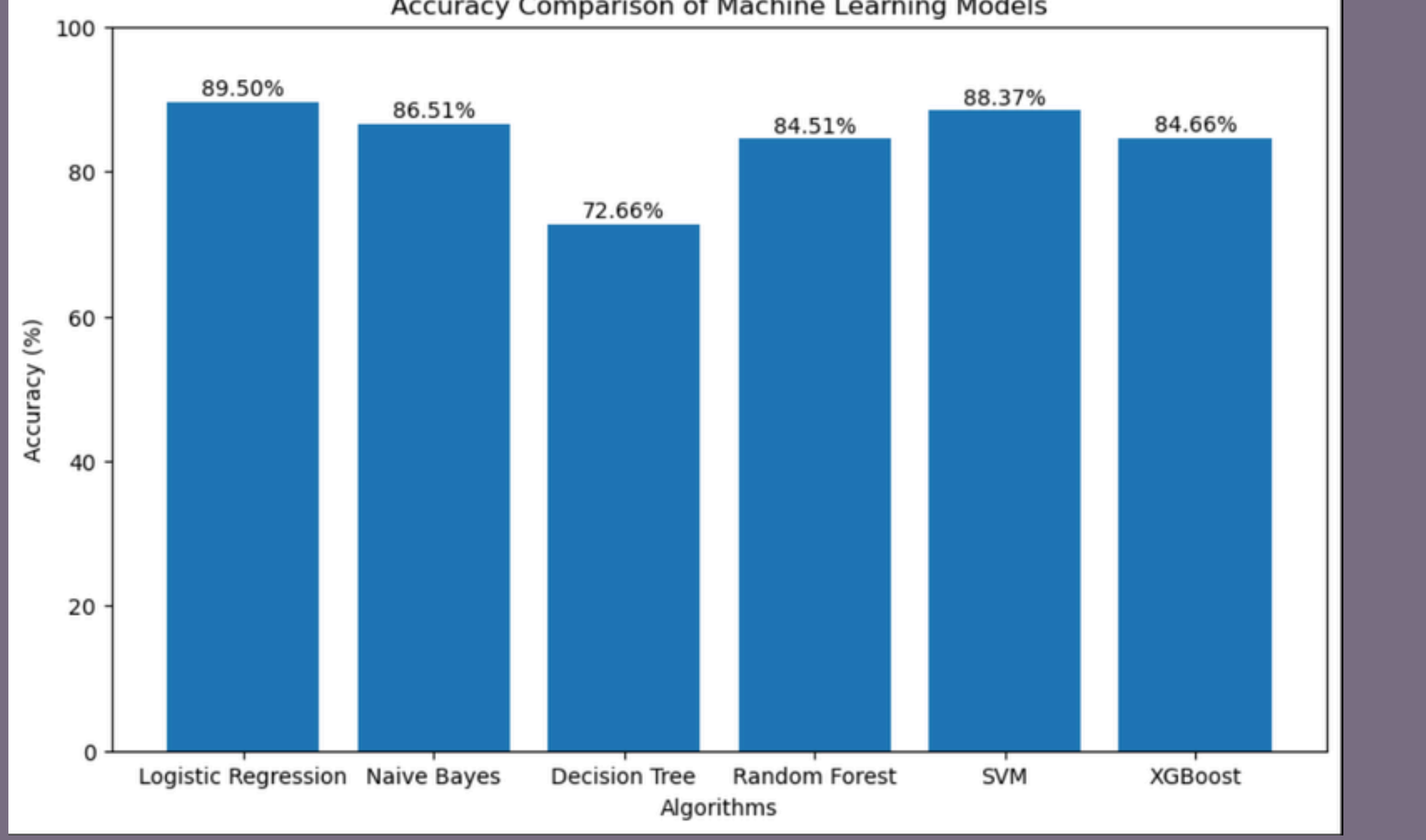




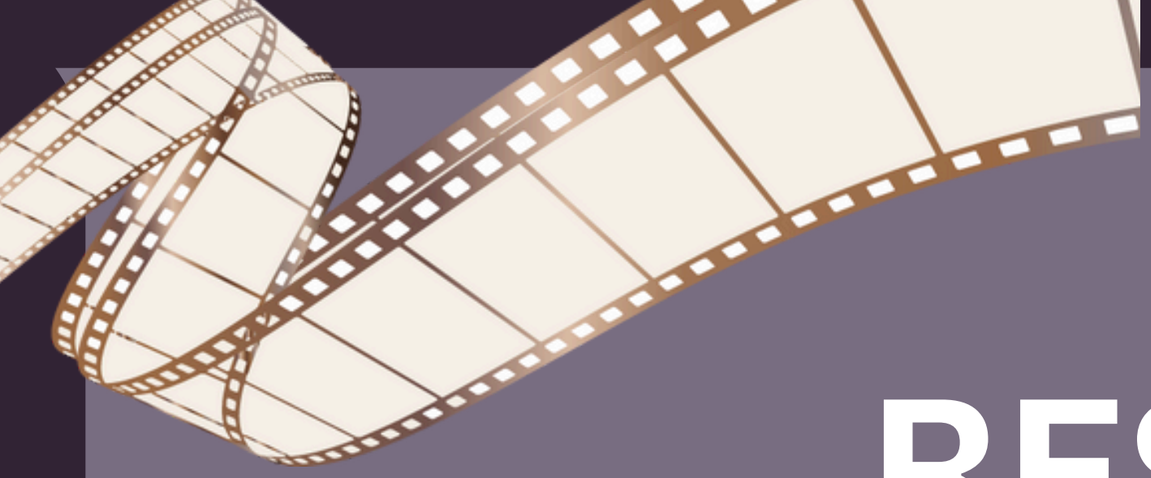


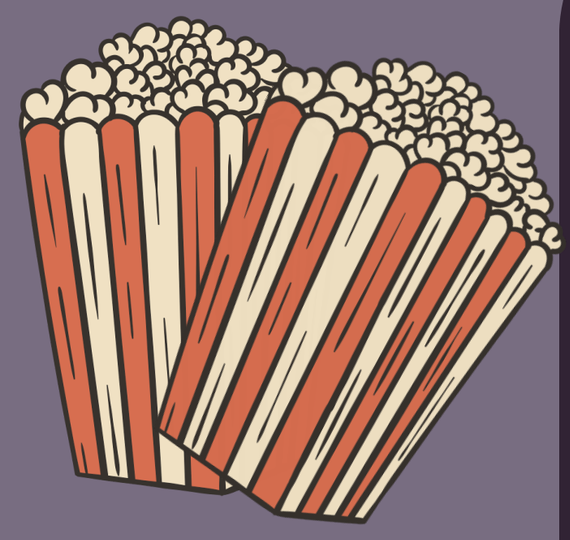
The XGBoost confusion matrix provides a visual breakdown of the model's predictive accuracy, highlighting its effectiveness in binary sentiment classification. The heatmap shows that the model successfully identified 4,041 negative reviews and 4,355 positive reviews, indicating a strong ability to capture the underlying sentiment of the movie review dataset. While there were 898 instances of false positives and 623 instances of false negatives, the high concentration of values along the main diagonal confirms the model's overall robustness. This visualization serves as a key performance indicator, demonstrating that the gradient boosting approach maintains a balanced and reliable classification rate across both target classes.

ACCURACY COMPARSION



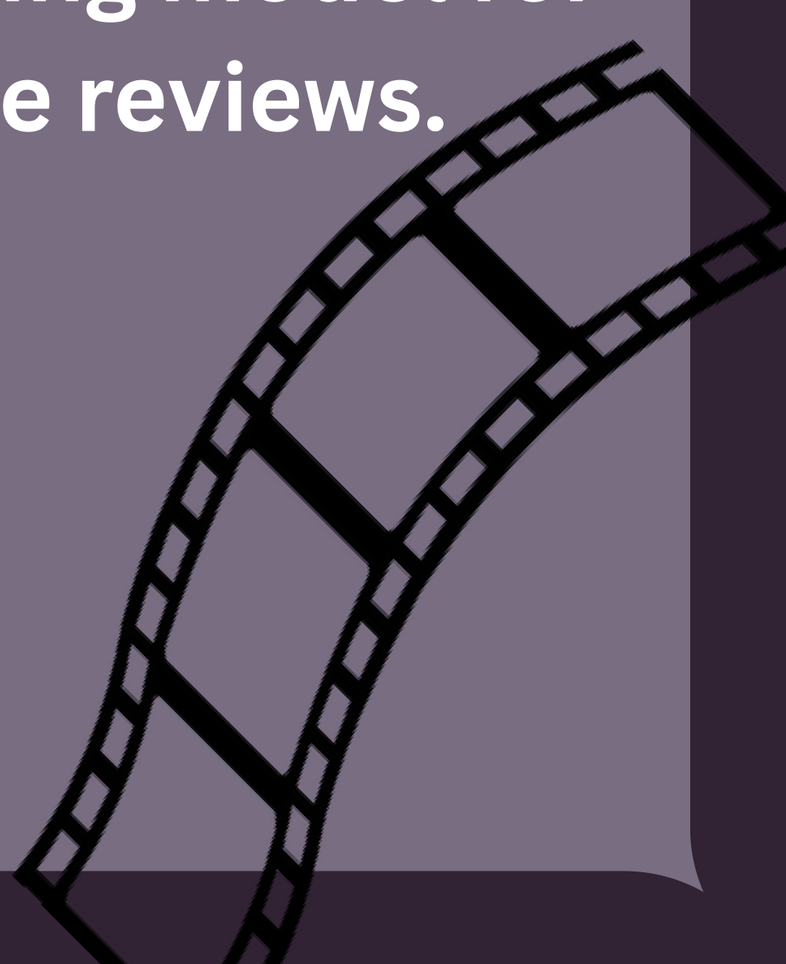
The above graph presents the accuracy comparison of different machine learning algorithms used for movie review sentiment classification. Among all the implemented models, Logistic Regression achieved the highest accuracy of 89.50%, followed closely by SVM with 88.37% accuracy. Naive Bayes also performed well with an accuracy of 86.51%, while Random Forest and XGBoost showed moderate performance. Decision Tree produced the lowest accuracy among all models. The comparison demonstrates that linear classification models such as Logistic Regression and SVM performed more effectively for textual sentiment analysis using TF-IDF features

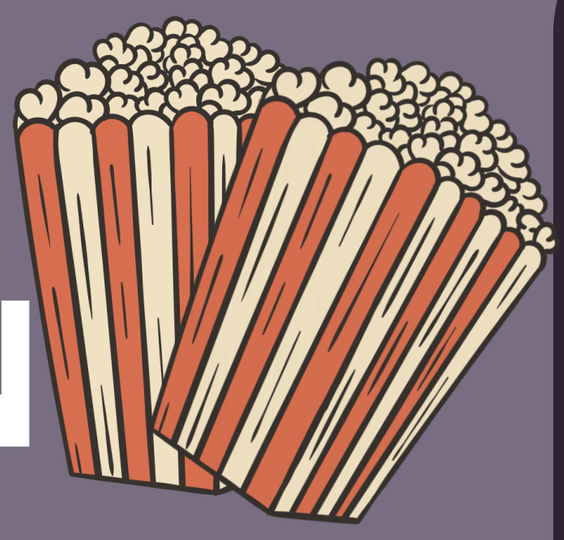




RESULT

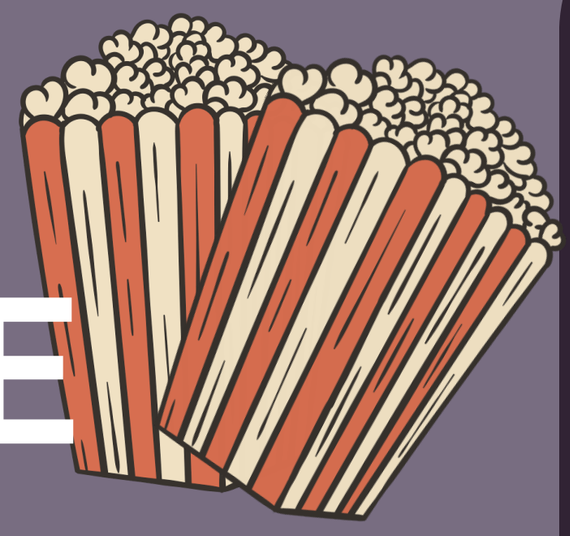
The Movie Review Sentiment Analysis project was successfully implemented using various Natural Language Processing and Machine Learning techniques. Text preprocessing methods such as cleaning, tokenization, stopword removal, stemming, slang handling, and TF-IDF feature extraction improved the quality of textual data and enhanced model performance. Multiple classification algorithms including Logistic Regression, Naive Bayes, Decision Tree, Random Forest, SVM, and XGBoost were trained and evaluated using performance metrics such as Accuracy Score, Classification Report, and Confusion Matrix. Among all the models, Logistic Regression achieved the highest accuracy score of 89.50%, making it the best-performing model for sentiment classification of movie reviews.





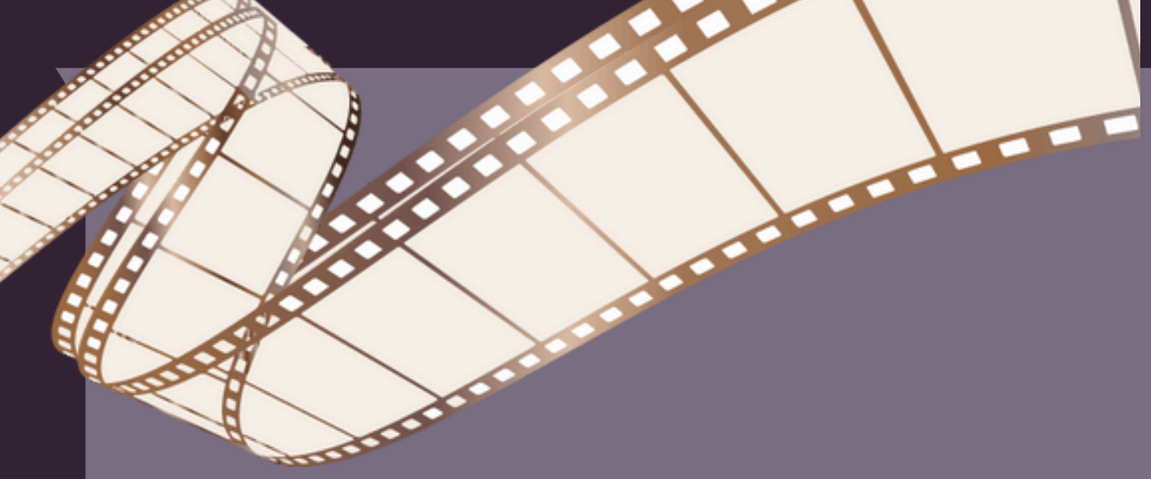
CONCLUSION

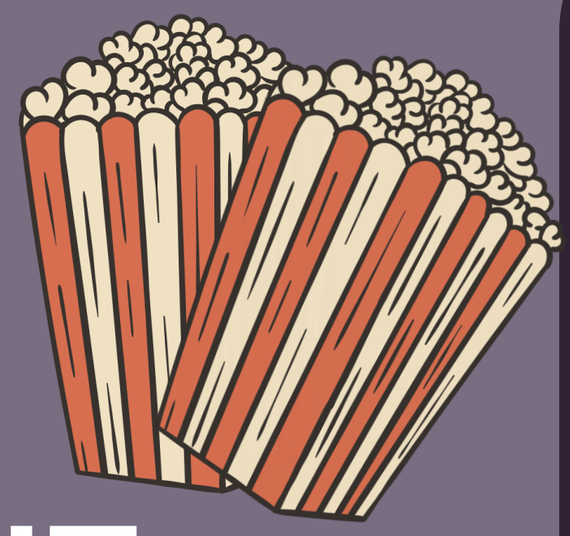
This project successfully demonstrated the application of Natural Language Processing and Machine Learning techniques for movie review sentiment analysis. Through effective text preprocessing and feature extraction, raw textual reviews were transformed into meaningful numerical representations suitable for machine learning models. Different classification algorithms were implemented and compared to analyze their performance in predicting positive and negative sentiments. The project provided practical knowledge of NLP concepts such as text cleaning, tokenization, stemming, TF-IDF vectorization, and sentiment classification while also improving understanding of machine learning model evaluation and comparison techniques. Overall, the project achieved its objective of building an efficient and accurate sentiment analysis system for classifying movie reviews.



FUTURE SCOPE

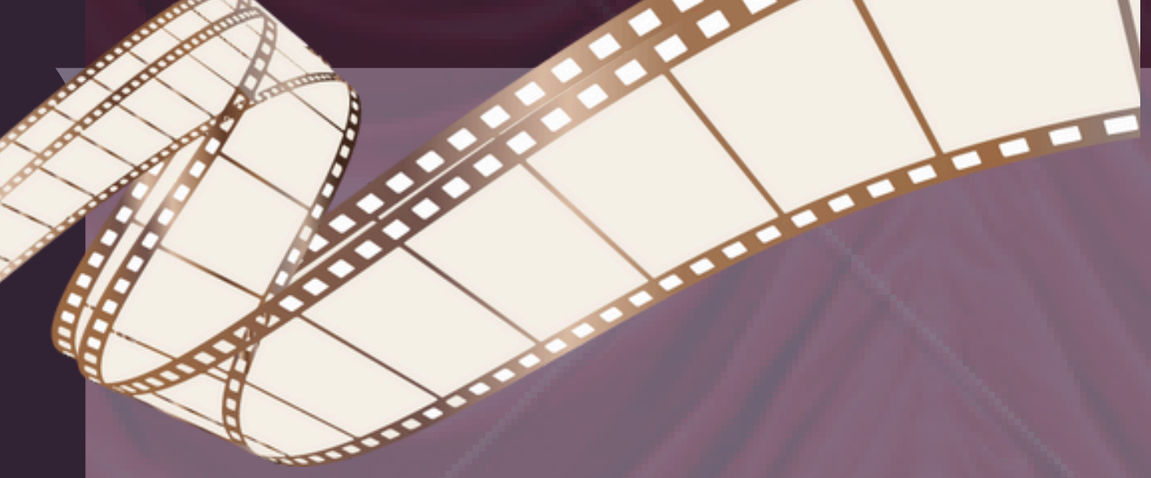
The future scope of this project includes improving the sentiment analysis system by using larger and more diverse movie review datasets and implementing advanced deep learning and Natural Language Processing techniques such as LSTM, RNN, and Transformer-based models like BERT. The project can also be extended to perform multi-class sentiment analysis, emotion detection, and real-time review prediction from online platforms. Additional improvements such as multilingual sentiment analysis, sarcasm detection, and deployment as a web application can further enhance the accuracy, efficiency, and practical usability of the system in real-world applications.

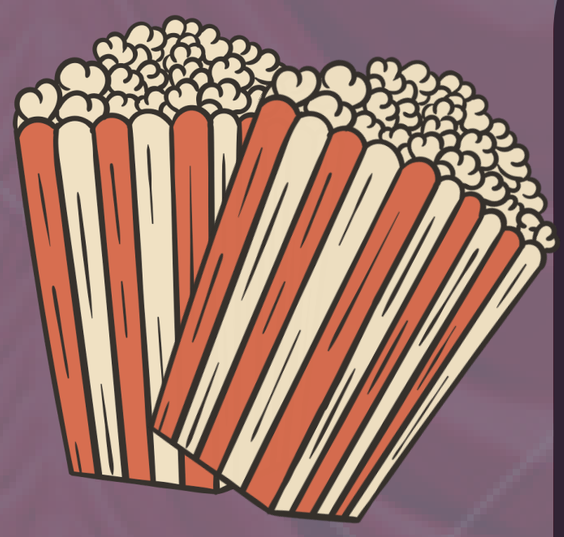




ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my teachers and mentors for their valuable guidance, support, and encouragement throughout the development of this NLP-based sentiment analysis project. Their continuous motivation helped me understand important concepts related to Natural Language Processing, Machine Learning, and text classification techniques. I am also thankful to various online resources, open-source Python libraries, and the movie review dataset providers that contributed significantly to the successful completion of this project.
Finally, I would like to thank everyone who directly or indirectly supported me during this work.





THE END

Thank You for Watching 🍿
Sentiment Analysis: The Final Review
Written & Directed By
Apoorva Sharma
Starring:
Machine Learning, NLP, and Movie Reviews

“Model Accuracy Approved ✓“
“Critics rated this project: Positive Sentiment”